

CSE145 – Introduction to Computing

Assignment 2: Turing Machines – Design, Analysis & Simulation

This assignment is individually personalized – read the Personalization section before you start anything else.

Name: Mohammed

ERP No.: 35791

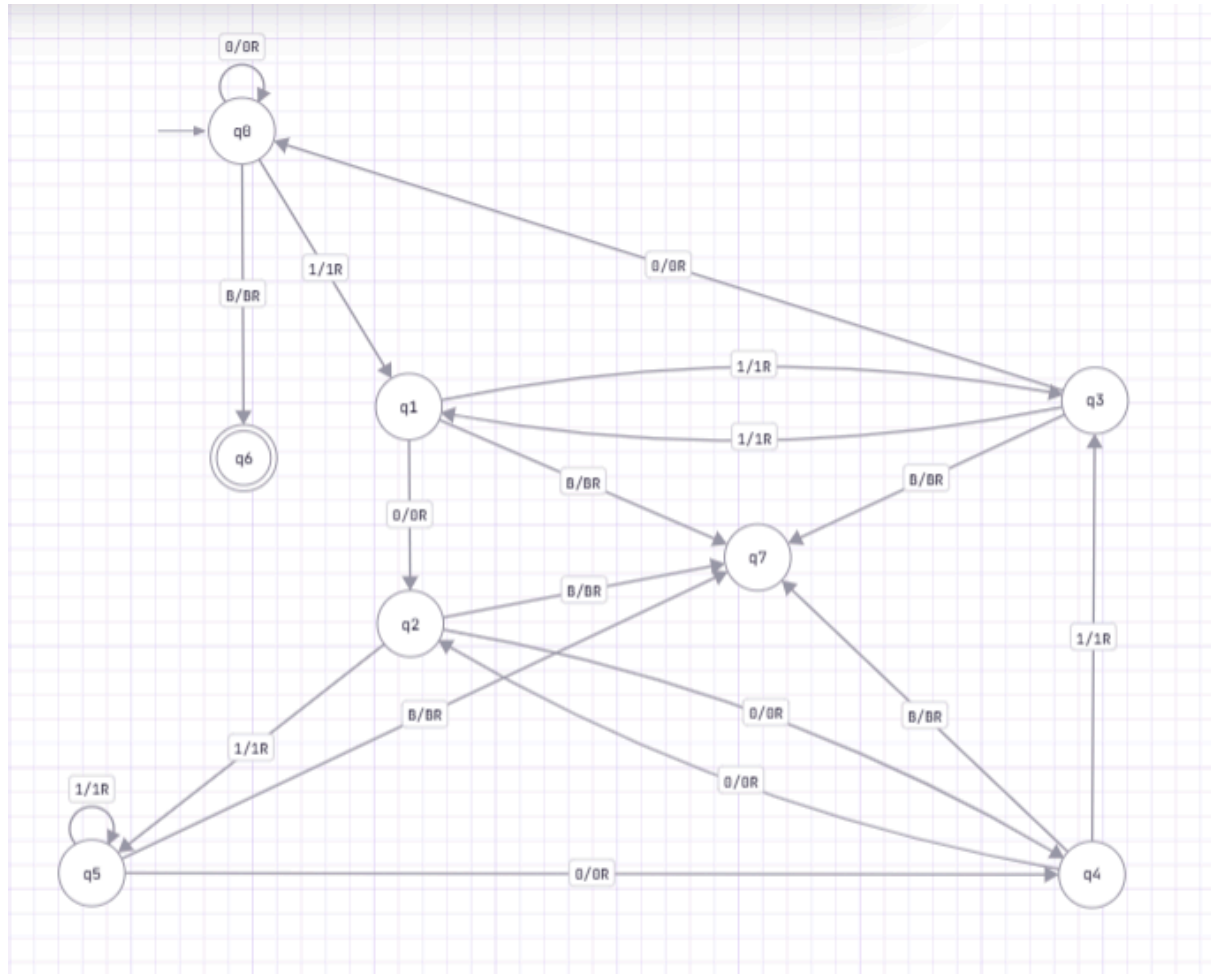
Personalization block (fill in – see below): R = 91 V = 3 B = “1011011” n = 3

Question 1:

Task A:

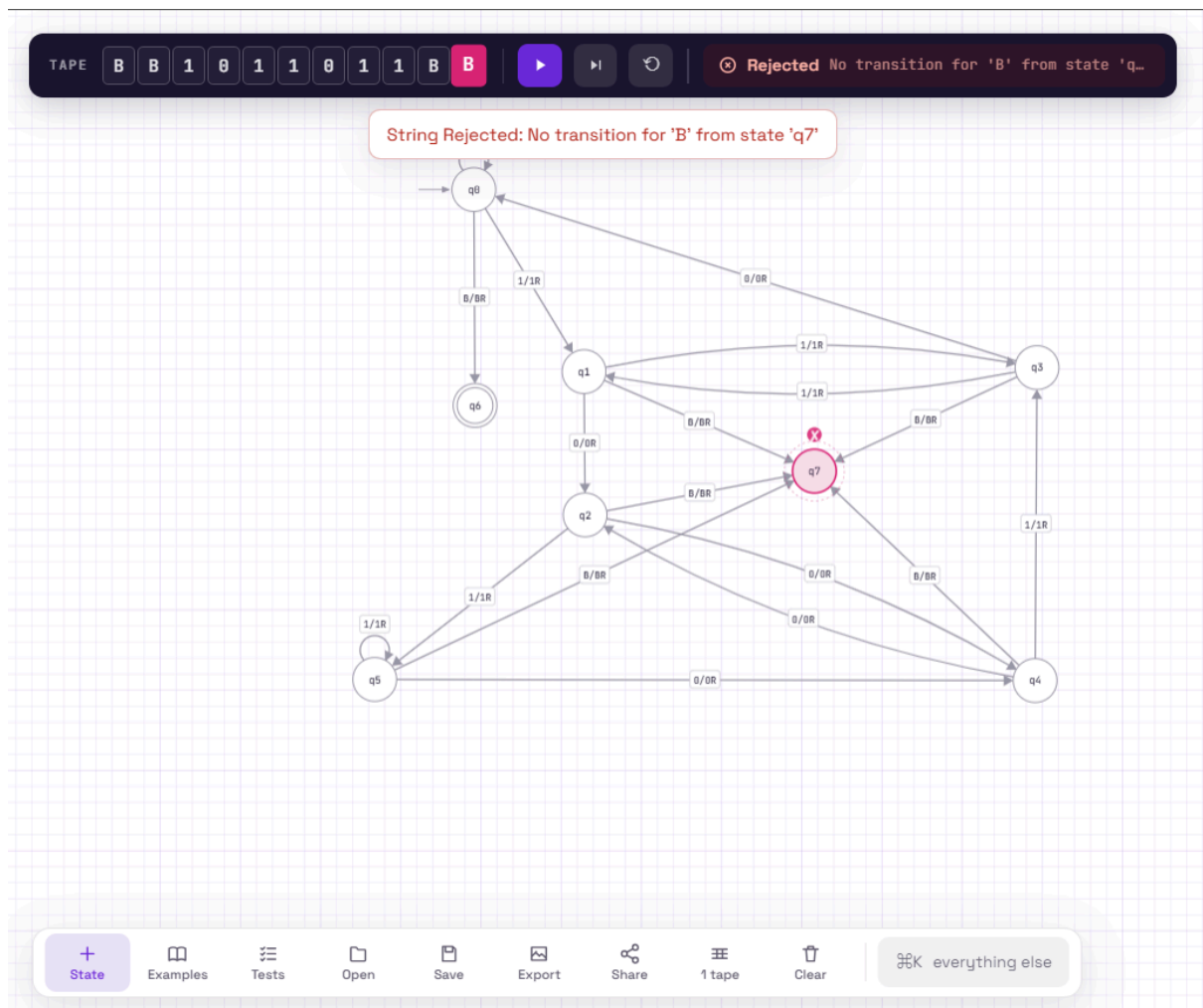
The Turing machine reads the tape from left to right, going from the most significant bit first, one bit at a time. The machine would work using six states, q_0 to q_5 ; these states represent the current remainder modulo 6. The transitions takes place using the formula $(\text{current remainder} * 2 + \text{new bit}) \% 6$. This just takes the remainder of the previous bit we calculated, multiplies it by 2, adds the new bit that we are reading and finally takes the modulo of it to find the remainder. The remainder tells which state to move on eg. remainder we are on q_3 and remainder is 5 it moves to q_5 . The machine halts when it encounters a blank symbol and checks its current state. If the current state is q_0 , it transitions to the accept state; otherwise, it rejects, meaning the divisibility was not successful.

Task B:



TaskC:

1011011 Rejected: [Evidence with personalized bit string B](#)



```

JSON: {"nodeMap":{"q0":
{"id":"q0","x":540.2000568369307,"y":-136.2866273169582
3},"q1":
{"id":"q1","x":691.3529955503166,"y":113.2593052066044
4},"q2":
{"id":"q2","x":692.8136968207627,"y":309.6540127525771},"
q3":
{"id":"q3","x":1312.6179953687847,"y":107.7067587685919},"
q4":
{"id":"q4","x":1310.0095336184652,"y":536.684228092994
1},"q5":
{"id":"q5","x":404.2511107767336,"y":535.1606530082432},
"q6":

```

```

{"id":"q6","x":541.8568133641608,"y":159.66774415074707},
"q7":
{"id":"q7","x":1007.2986353568999,"y":249.6214300647968
7}},
"transitionMap":{
"q0":
[{"label":"1/1R","sourceid":"q0","targetid":"q1"},
{"label":"0/OR","sourceid":"q0","targetid":"q0"},
{"label":"B/BR","sourceid":"q0","targetid":"q6"}],
"q1":
[{"label":"0/OR","sourceid":"q1","targetid":"q2"},
{"label":"1/1R","sourceid":"q1","targetid":"q3"},
{"label":"B/BR","sourceid":"q1","targetid":"q7"}],
"q2":
[{"label":"0/OR","sourceid":"q2","targetid":"q4"},
{"label":"1/1R","sourceid":"q2","targetid":"q5"},
{"label":"B/BR","sourceid":"q2","targetid":"q7"}],
"q3":
[{"label":"0/OR","sourceid":"q3","targetid":"q0"},
{"label":"1/1R","sourceid":"q3","targetid":"q1"},
{"label":"B/BR","sourceid":"q3","targetid":"q7"}],
"q4":
[{"label":"0/OR","sourceid":"q4","targetid":"q2"},
{"label":"1/1R","sourceid":"q4","targetid":"q3"},
{"label":"B/BR","sourceid":"q4","targetid":"q7"}],
"q5":
[{"label":"0/OR","sourceid":"q5","targetid":"q4"},
{"label":"1/1R","sourceid":"q5","targetid":"q5"},
{"label":"B/BR","sourceid":"q5","targetid":"q7"}]],
"finalNodes":
["q6"],
"nodeNum":8,
"automataType":"TM",
"inputString":"1011011",
"stageProps":
{"x":358.9128514104228,"y":212.9175359265078,"scale":0.6655000000000002,
"draggable":true},
"stateOutput":
{},"tapeMode":"single",
"inputString2":""}

```

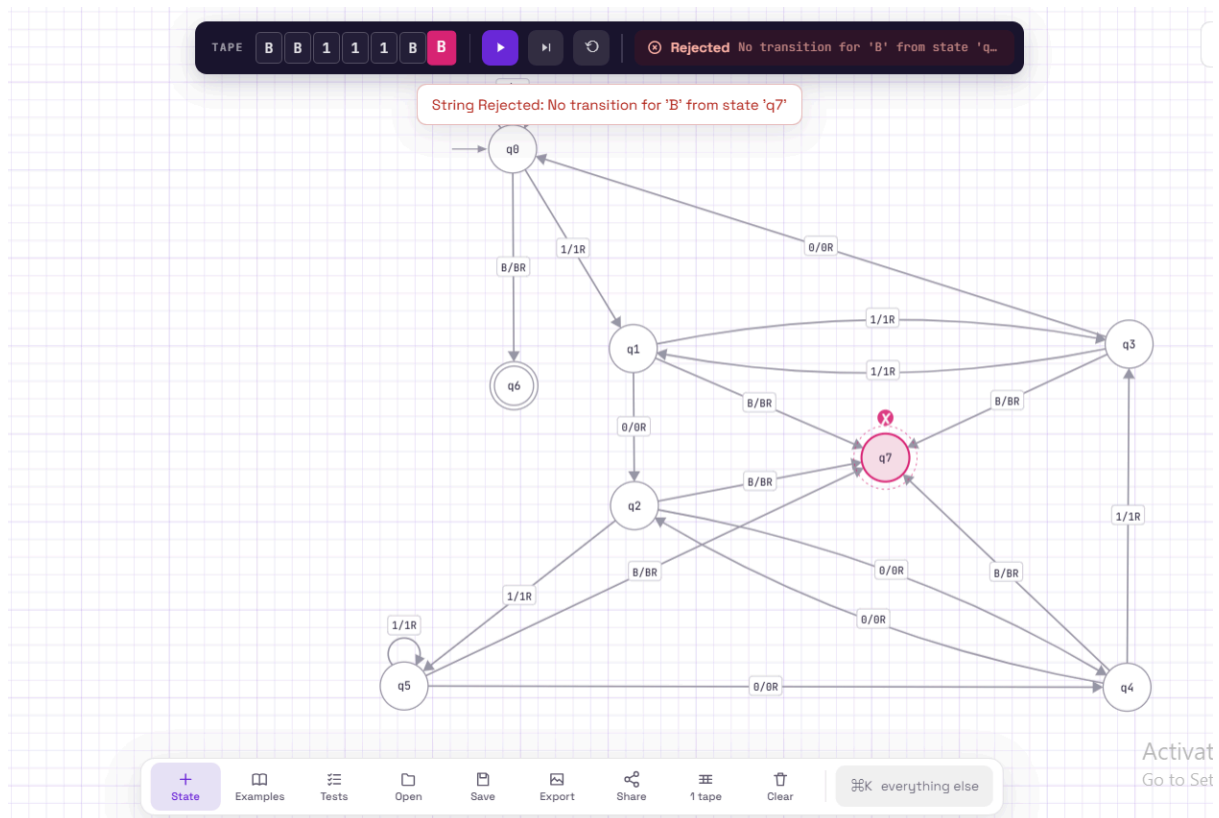
110 Accepted: [Q1 Task C evidence with string of choice that IS divisible by M](#)


```

7}}, "transitionMap": {"q0":
[{"label": "1/1R", "sourceid": "q0", "targetid": "q1"},
{"label": "0/OR", "sourceid": "q0", "targetid": "q0"},
{"label": "B/BR", "sourceid": "q0", "targetid": "q6"}], "q1":
[{"label": "0/OR", "sourceid": "q1", "targetid": "q2"},
{"label": "1/1R", "sourceid": "q1", "targetid": "q3"},
{"label": "B/BR", "sourceid": "q1", "targetid": "q7"}], "q2":
[{"label": "0/OR", "sourceid": "q2", "targetid": "q4"},
{"label": "1/1R", "sourceid": "q2", "targetid": "q5"},
{"label": "B/BR", "sourceid": "q2", "targetid": "q7"}], "q3":
[{"label": "0/OR", "sourceid": "q3", "targetid": "q0"},
{"label": "1/1R", "sourceid": "q3", "targetid": "q1"},
{"label": "B/BR", "sourceid": "q3", "targetid": "q7"}], "q4":
[{"label": "0/OR", "sourceid": "q4", "targetid": "q2"},
{"label": "1/1R", "sourceid": "q4", "targetid": "q3"},
{"label": "B/BR", "sourceid": "q4", "targetid": "q7"}], "q5":
[{"label": "0/OR", "sourceid": "q5", "targetid": "q4"},
{"label": "1/1R", "sourceid": "q5", "targetid": "q5"},
{"label": "B/BR", "sourceid": "q5", "targetid": "q7"}]}, "finalNodes":
["q6"], "nodeNum": 8, "automataType": "TM", "inputString": "110", "stageProps":
{"x": 358.9128514104228, "y": 212.9175359265078, "scale": 0.6655000000000002, "draggable": true}, "stateOutput":
{}, "tapeMode": "single", "inputString2": ""}

```

111 Rejected: [Q1 Task C evidence with string of choice that IS not divisible by M](#)



```
JSON: {"nodeMap":{"q0":
{"id":"q0","x":540.2000568369307,"y":-136.2866273169582
3},"q1":
{"id":"q1","x":691.3529955503166,"y":113.2593052066044
4},"q2":
{"id":"q2","x":692.8136968207627,"y":309.6540127525771},"
q3":
{"id":"q3","x":1312.6179953687847,"y":107.7067587685919},
q4":
{"id":"q4","x":1310.0095336184652,"y":536.684228092994
1},"q5":
{"id":"q5","x":404.2511107767336,"y":535.1606530082432},
"q6":
{"id":"q6","x":541.8568133641608,"y":159.66774415074707}
,"q7":
{"id":"q7","x":1007.2986353568999,"y":249.6214300647968
7}},"transitionMap":{"q0":
```

```
[{"label":"1/1R","sourceid":"q0","targetid":"q1"},
{"label":"0/OR","sourceid":"q0","targetid":"q0"},
{"label":"B/BR","sourceid":"q0","targetid":"q6"}], "q1":
[{"label":"0/OR","sourceid":"q1","targetid":"q2"},
{"label":"1/1R","sourceid":"q1","targetid":"q3"},
{"label":"B/BR","sourceid":"q1","targetid":"q7"}], "q2":
[{"label":"0/OR","sourceid":"q2","targetid":"q4"},
{"label":"1/1R","sourceid":"q2","targetid":"q5"},
{"label":"B/BR","sourceid":"q2","targetid":"q7"}], "q3":
[{"label":"0/OR","sourceid":"q3","targetid":"q0"},
{"label":"1/1R","sourceid":"q3","targetid":"q1"},
{"label":"B/BR","sourceid":"q3","targetid":"q7"}], "q4":
[{"label":"0/OR","sourceid":"q4","targetid":"q2"},
{"label":"1/1R","sourceid":"q4","targetid":"q3"},
{"label":"B/BR","sourceid":"q4","targetid":"q7"}], "q5":
[{"label":"0/OR","sourceid":"q5","targetid":"q4"},
{"label":"1/1R","sourceid":"q5","targetid":"q5"},
{"label":"B/BR","sourceid":"q5","targetid":"q7"}]], "finalNodes":
["q6"], "nodeNum":8, "automataType":"TM", "inputString":"111", "stageProps":
{"x":358.9128514104228, "y":212.9175359265078, "scale":0.6655000000000002, "draggable":true}, "stateOutput":
{}, "tapeMode":"single", "inputString2":""}
```

TaskD:

The current machine uses six states to deal with the six different possible remainders (0 to 5). If M were to change to some other number, the number of states would also need to be changed to deal with the number of possible remainders of the new M . Also

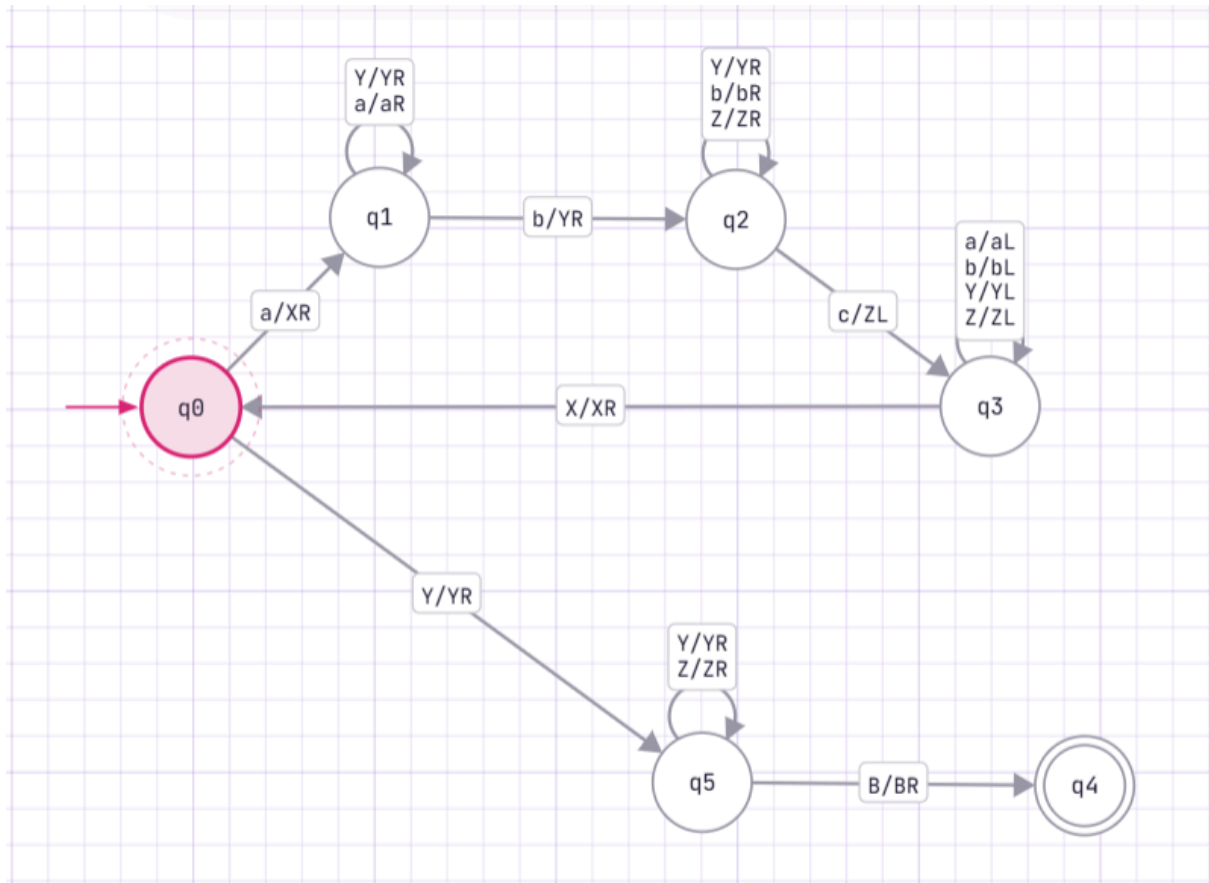
the current formula to calculate remainder is $(\text{current remainder} * 2 + \text{new bit}) \% 6$ which is specific to $M = 6$ only changing M would require a new formula logic else states would point to incorrect states. So, in short, the number of states and the arrows pointing would need to be changed.

Question 2:

TaskA:

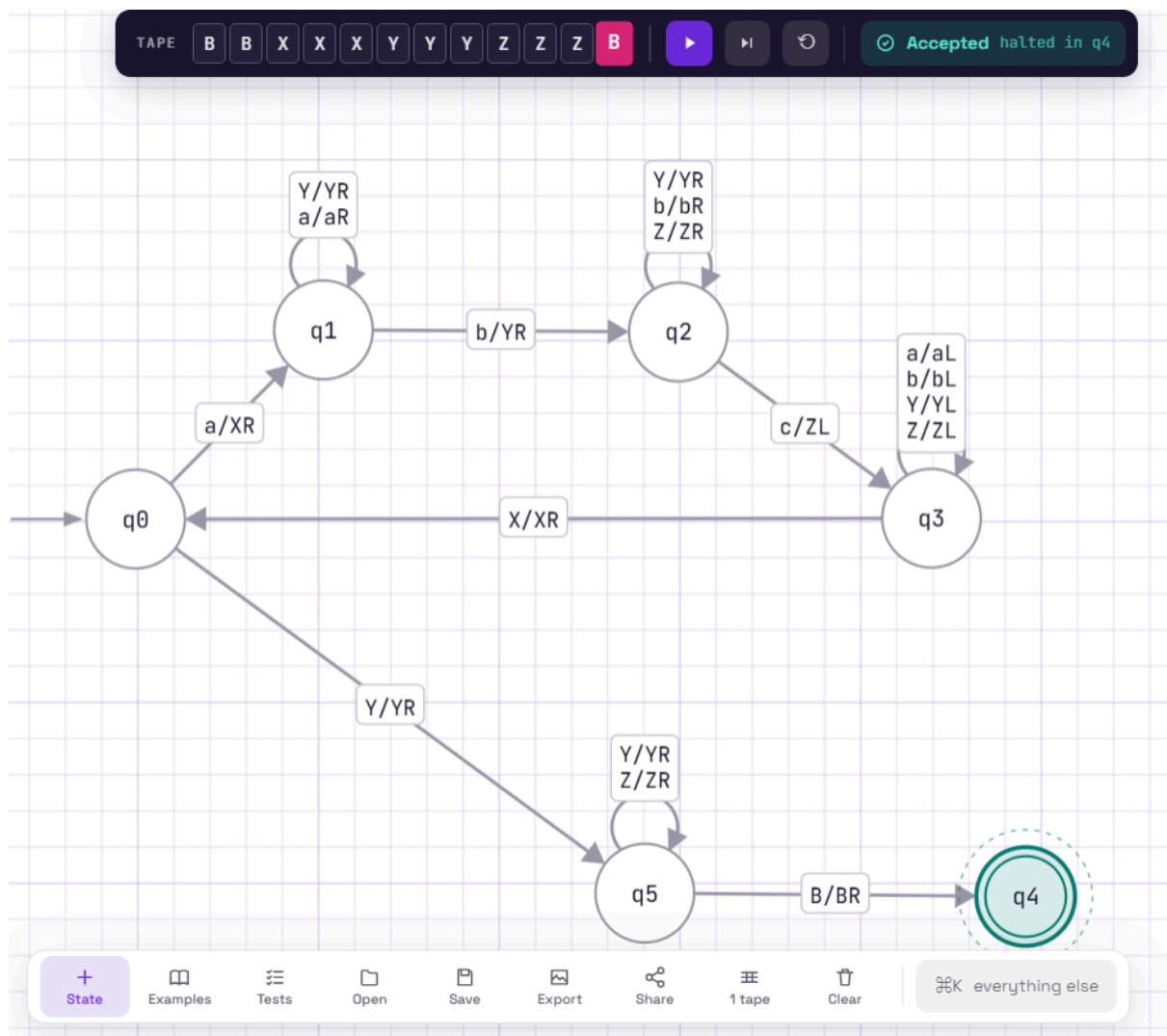
The tape moves from left to right, crossing each character block to ensure its counts match. It starts from the left, reads its first 'a' and marks it with an X, then moves right, skipping over until it reaches 'b' and replaces it with Y, then moves right, skipping over until it reaches 'c' and replaces it with Z. After marking a 'c', it rewinds leftwards all the way back to the first X and repeats the whole process. Once all the 'a's are converted to X, the machine runs for the final time as the number of $a=b=c$. As it runs for the final time, if it encounters no more 'b's or 'c's and only Y and Z are encountered until it reaches a blank, then it moves to the accept state.

TaskB:



TaskC:

aaabbbccc accept: [n = your personalized n repetitions](#)



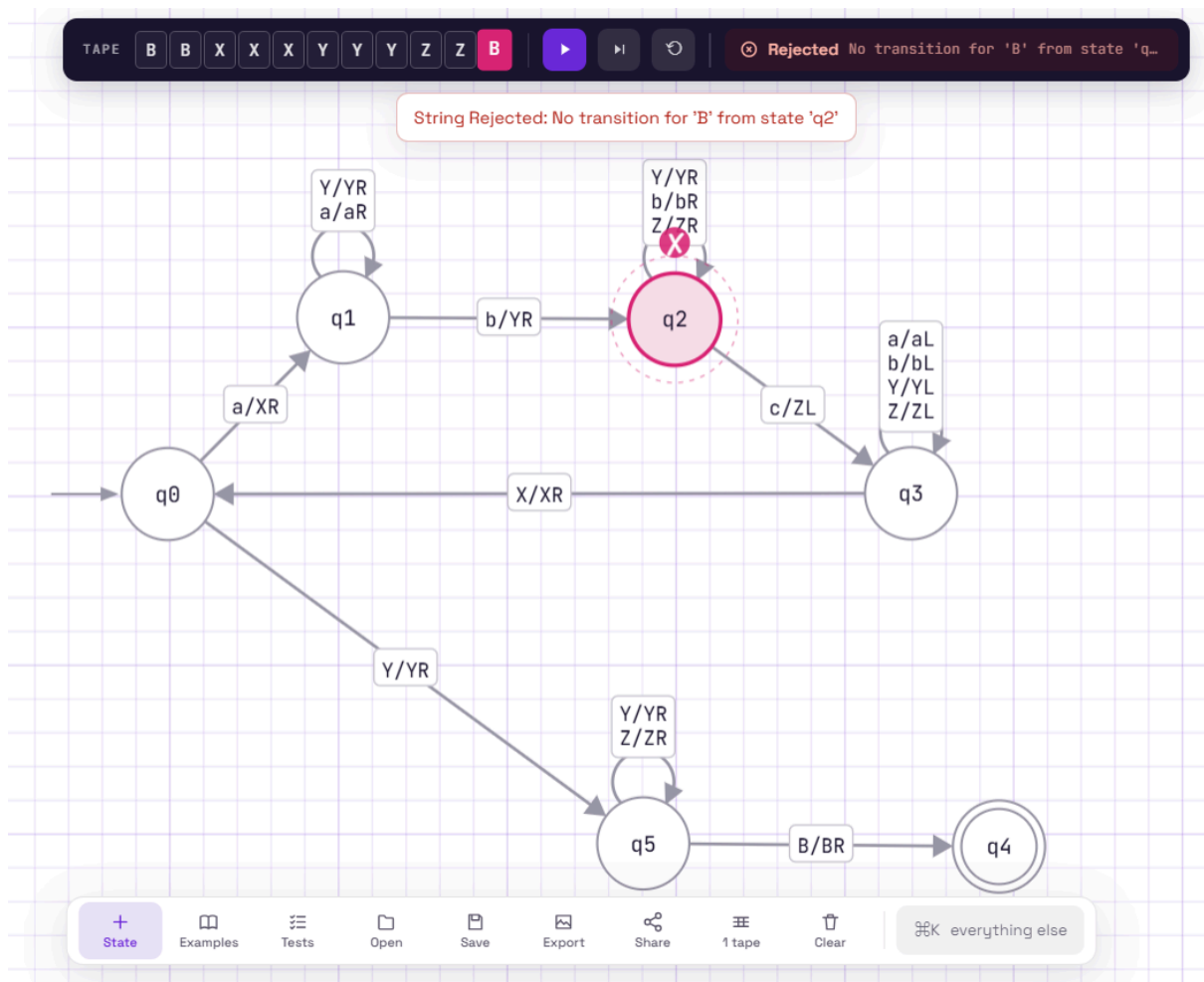
JSON: {"nodeMap":{"q0":
{"id":"q0","x":218.6328552607979,"y":218.4989224699519},
"q1":
{"id":"q1","x":332.97375354190126,"y":103.4505551886054
2},"q2":
{"id":"q2","x":549.0051614555043,"y":104.7055419293142},
"q3":
{"id":"q3","x":703.1066517887094,"y":217.99136879120158},
"q4":
{"id":"q4","x":760.4278770328999,"y":448.02210403718703}
,"q5":

```

{"id":"q5","x":528.5809806184797,"y":445.97689607462763
},"transitionMap":{"q0":
[{"label":"a/XR","sourceid":"q0","targetid":"q1"},
{"label":"Y/YR","sourceid":"q0","targetid":"q5"}],"q1":
[{"label":"b/YR","sourceid":"q1","targetid":"q2"},
{"label":"Y/YR,a/aR","sourceid":"q1","targetid":"q1"}],"q2":
[{"label":"c/ZL","sourceid":"q2","targetid":"q3"},
{"label":"Y/YR,b/bR,Z/ZR","sourceid":"q2","targetid":"q2"}],"
q3":
[{"label":"a/aL,b/bL,Y/YL,Z/ZL","sourceid":"q3","targetid":"q3
"},{"label":"X/XR","sourceid":"q3","targetid":"q0"}],"q5":
[{"label":"Y/YR,Z/ZR","sourceid":"q5","targetid":"q5"},
{"label":"B/BR","sourceid":"q5","targetid":"q4"}]},"finalNode
s":
["q4"],"nodeNum":6,"automataType":"TM","inputString":"aa
abbbccc","stageProps":
{"x":315.34861574525695,"y":102.7712899607102,"scale":0.
8705165431605937,"draggable":true},"stateOutput":
{},"tapeMode":"single","inputString2":""}

```

aaabbbccc reject: [string with one symbol added or removed so it falls just outside the language](#)



```

JSON: {"nodeMap":{"q0":
{"id":"q0","x":218.6328552607979,"y":218.4989224699519},
"q1":
{"id":"q1","x":332.97375354190126,"y":103.4505551886054
2},"q2":
{"id":"q2","x":549.0051614555043,"y":104.7055419293142},
"q3":
{"id":"q3","x":703.1066517887094,"y":217.99136879120158},
"q4":
{"id":"q4","x":760.4278770328999,"y":448.02210403718703},
"q5":
{"id":"q5","x":528.5809806184797,"y":445.97689607462763
}}, "transitionMap":{"q0":
[{"label":"a/XR","sourceid":"q0","targetid":"q1"},

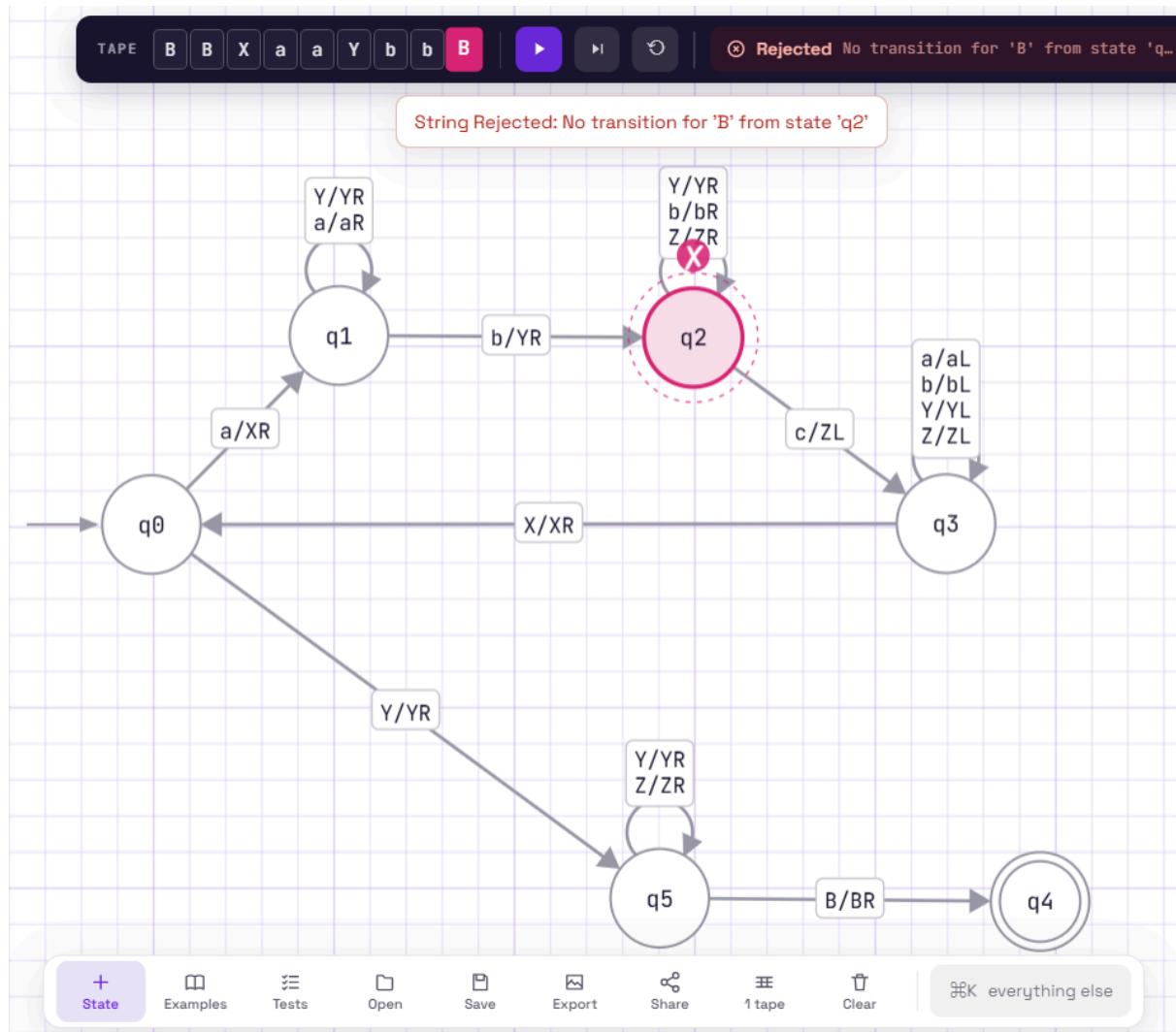
```

```

{"label":"Y/YR","sourceid":"q0","targetid":"q5"}], "q1":
[{"label":"b/YR","sourceid":"q1","targetid":"q2"},
{"label":"Y/YR,a/aR","sourceid":"q1","targetid":"q1"}], "q2":
[{"label":"c/ZL","sourceid":"q2","targetid":"q3"},
{"label":"Y/YR,b/bR,Z/ZR","sourceid":"q2","targetid":"q2"}], "
q3":
[{"label":"a/aL,b/bL,Y/YL,Z/ZL","sourceid":"q3","targetid":"q3
"}, {"label":"X/XR","sourceid":"q3","targetid":"q0"}], "q5":
[{"label":"Y/YR,Z/ZR","sourceid":"q5","targetid":"q5"},
{"label":"B/BR","sourceid":"q5","targetid":"q4"}]], "finalNode
s":
["q4"], "nodeNum":6, "automataType":"TM", "inputString":"aa
abbbcc", "stageProps":
{"x":244.4955284844949, "y":113.88092878191941, "scale":1
.07179440500000008, "draggable":true}, "stateOutput":
{}, "tapeMode":"single", "inputString2":""}

```

aaabbb reject: [one further string of my choice](#)



```

JSON: {"nodeMap":{"q0":
{"id":"q0","x":218.6328552607979,"y":218.4989224699519},
"q1":
{"id":"q1","x":332.97375354190126,"y":103.4505551886054
2},"q2":
{"id":"q2","x":549.0051614555043,"y":104.7055419293142},
"q3":
{"id":"q3","x":703.1066517887094,"y":217.99136879120158},
"q4":
{"id":"q4","x":760.4278770328999,"y":448.02210403718703}
,"q5":
{"id":"q5","x":528.5809806184797,"y":445.97689607462763
}}, "transitionMap":{"q0":

```

```
[{"label":"a/XR","sourceid":"q0","targetid":"q1"},
{"label":"Y/YR","sourceid":"q0","targetid":"q5"}], "q1":
[{"label":"b/YR","sourceid":"q1","targetid":"q2"},
{"label":"Y/YR,a/aR","sourceid":"q1","targetid":"q1"}], "q2":
[{"label":"c/ZL","sourceid":"q2","targetid":"q3"},
{"label":"Y/YR,b/bR,Z/ZR","sourceid":"q2","targetid":"q2"}], "
q3":
[{"label":"a/aL,b/bL,Y/YL,Z/ZL","sourceid":"q3","targetid":"q3
"}, {"label":"X/XR","sourceid":"q3","targetid":"q0"}], "q5":
[{"label":"Y/YR,Z/ZR","sourceid":"q5","targetid":"q5"},
{"label":"B/BR","sourceid":"q5","targetid":"q4"}]], "finalNode
s":
["q4"], "nodeNum":6, "automataType":"TM", "inputString":"aa
abbb", "stageProps":
{"x":244.4955284844949, "y":113.88092878191941, "scale":1
.07179440500000008, "draggable":true}, "stateOutput":
{}, "tapeMode":"single", "inputString2":""}
```

TaskD:

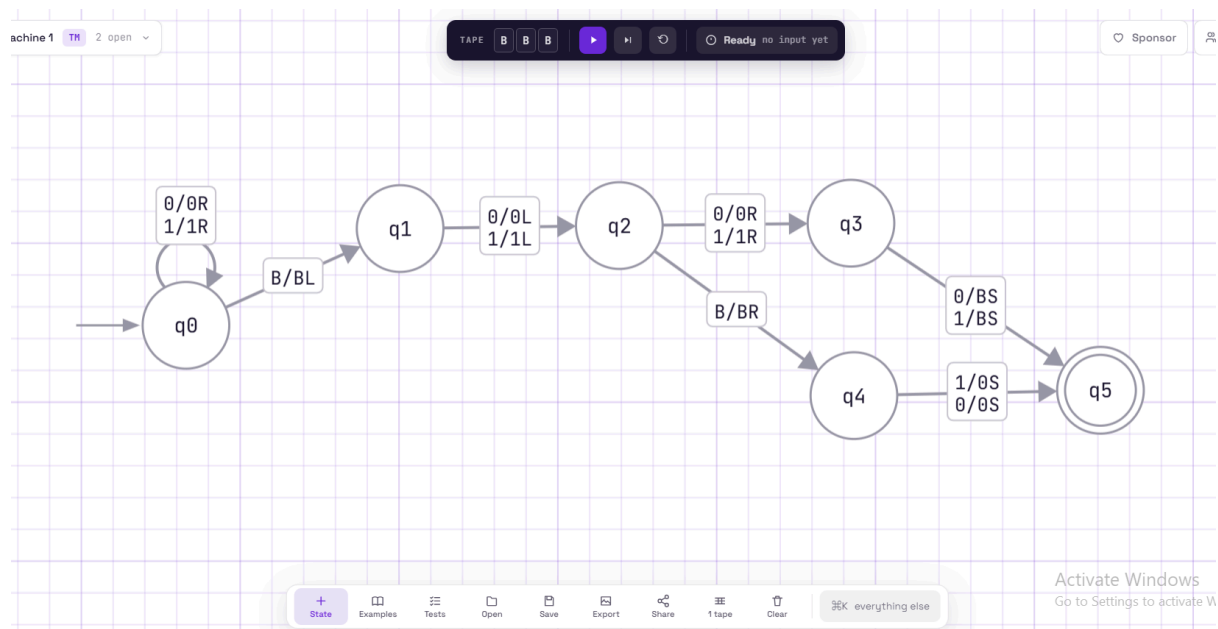
A off by one error could occur when a machine transitions to accept state prematurely where it doesn't check if it has any more 'c' remaining where it could accept a string as aaabbbcccc. To solve this I used a verification state where after the q0 checks all 'a' are changed to X thus it could be assumed all 'b' and 'c' are changed to Y and Z respectively but I still checked by running through the string to confirm if it reaches the blank without any extra 'c'.

Question 3:

TaskA:

The machine reads the tape from left to right, bit by bit, leaving all 0s and 1s unchanged until it encounters the first blank symbol at the end of the string. When it reaches the blank on the right side, it must change its direction to the left; then it just checks the least significant bit but doesn't change anything and moves to the next bit. If the bit is not a blank, it changes direction again and changes the next bit, which would be the least significant bit, to a blank. If, in case we initially found a blank, then it would still change its direction to the right but change the next bit (least significant bit) to 0; this would allow us to tackle cases where the string is only one bit.

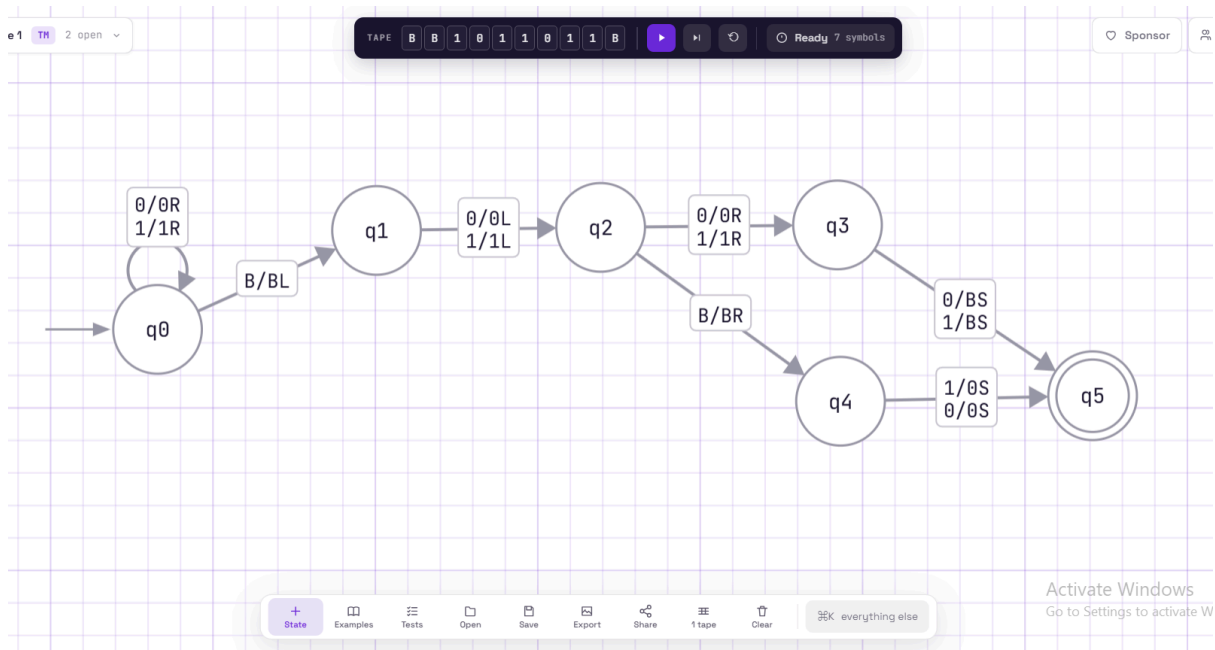
TaskB:



TaskC:

1011011: [personalized bit-string B](#)

Before:



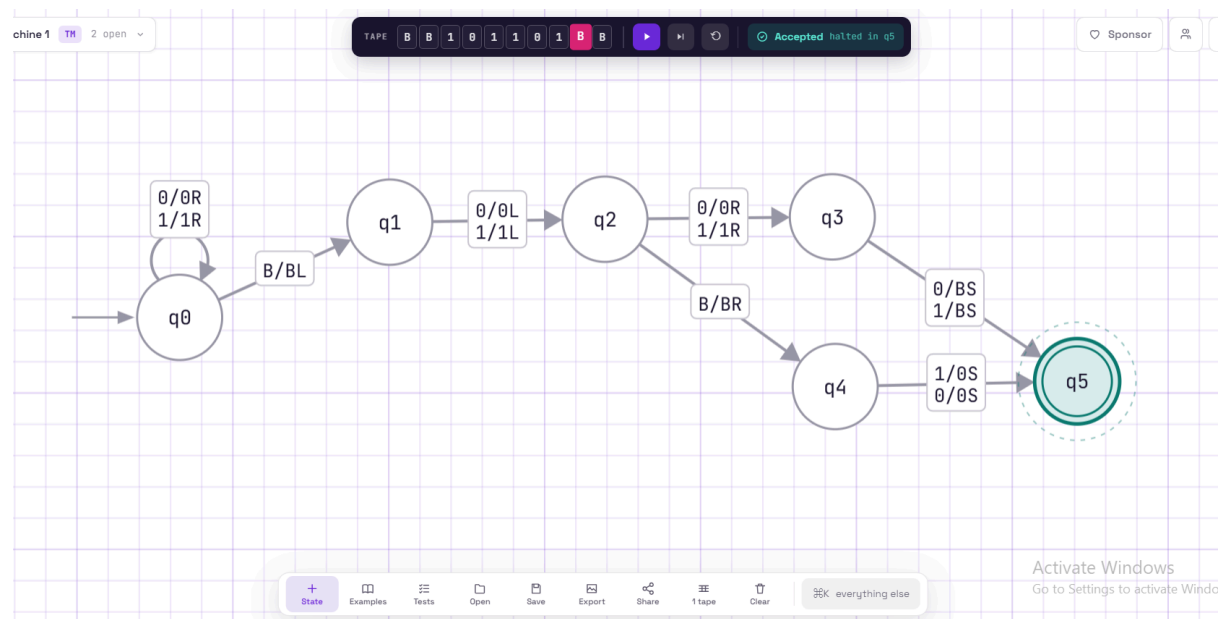
```
JSON: {"nodeMap":{"q0":
{"id":"q0","x":218.6328552607979,"y":218.4989224699519},
"q1":
{"id":"q1","x":332.97375354190126,"y":103.4505551886054
2},"q2":
{"id":"q2","x":549.0051614555043,"y":104.7055419293142},
"q3":
{"id":"q3","x":703.1066517887094,"y":217.99136879120158},
"q4":
{"id":"q4","x":760.4278770328999,"y":448.02210403718703}
,"q5":
{"id":"q5","x":528.5809806184797,"y":445.97689607462763
}}, "transitionMap":{"q0":
[{"label":"a/XR","sourceid":"q0","targetid":"q1"},
{"label":"Y/YR","sourceid":"q0","targetid":"q5"}],"q1":
[{"label":"b/YR","sourceid":"q1","targetid":"q2"},
{"label":"Y/YR,a/aR","sourceid":"q1","targetid":"q1"}],"q2":
[{"label":"c/ZL","sourceid":"q2","targetid":"q3"},
```

```

{"label":"Y/YR,b/bR,Z/ZR","sourceid":"q2","targetid":"q2"}],
q3":
[{"label":"a/aL,b/bL,Y/YL,Z/ZL","sourceid":"q3","targetid":"q3"},
{"label":"X/XR","sourceid":"q3","targetid":"q0"}],q5":
[{"label":"Y/YR,Z/ZR","sourceid":"q5","targetid":"q5"},
{"label":"B/BR","sourceid":"q5","targetid":"q4"}]],finalNodes":
["q4"],"nodeNum":6,"automataType":"TM","inputString":"aa
abbb","stageProps":
{"x":244.4955284844949,"y":113.88092878191941,"scale":1
.07179440500000008,"draggable":true},"stateOutput":
{},"tapeMode":"single","inputString2":""}

```

After:

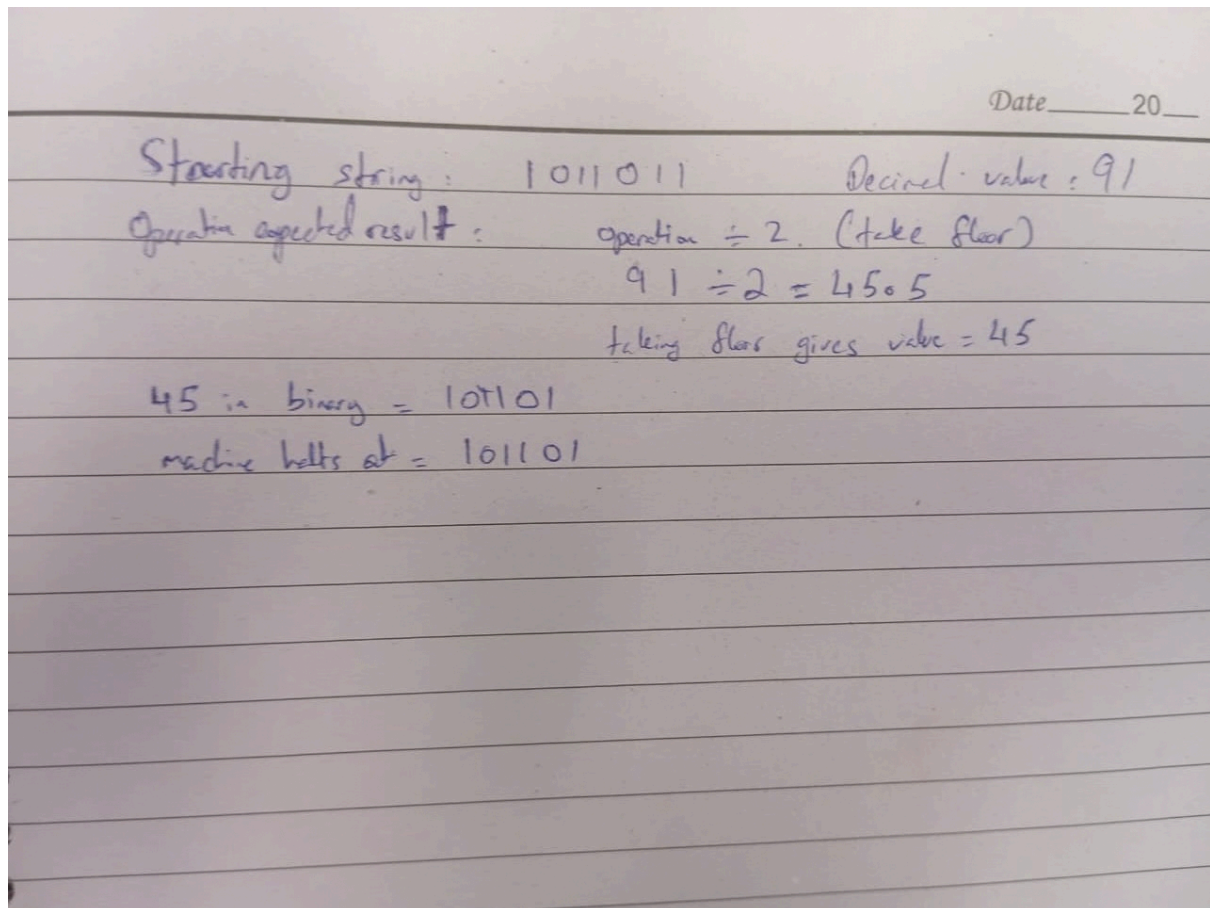


```

JSON: {"nodeMap":{"q0":
{"id":"q0","x":402.4279488470676,"y":166.8307845499095
7},"q1":
{"id":"q1","x":550.7533967796007,"y":99.87923421352778},"
q2":
{"id":"q2","x":702.5710953940167,"y":97.80821981664928},"

```

```
q3":
{"id":"q3","x":863.0147876077398,"y":96.14902761242507},
q4":
{"id":"q4","x":865.0509921517862,"y":215.4662620580540
4},"q5":
{"id":"q5","x":1035.803117459444,"y":211.7550957177448}},
transitionMap":{"q0":
[{"label":"B/BL","sourceid":"q0","targetid":"q1"},
{"label":"O/OR,1/1R","sourceid":"q0","targetid":"q0"}],"q1":
[{"label":"O/OL,1/1L","sourceid":"q1","targetid":"q2"}],"q2":
[{"label":"O/OR,1/1R","sourceid":"q2","targetid":"q3"},
{"label":"B/BR","sourceid":"q2","targetid":"q4"}],"q4":
[{"label":"1/OS,0/OS","sourceid":"q4","targetid":"q5"}],"q3":
[{"label":"O/BS,1/BS","sourceid":"q3","targetid":"q5"}]},"final
Nodes":
["q5"],"nodeNum":6,"automataType":"TM","inputString":"10
11011","stageProps":
{"x":-332.5516109275379,"y":85.6802802677148,"scale":1.5
692141883605017,"draggable":true},"stateOutput":
{},"tapeMode":"single","inputString2":""}
```



TaskD:

The leftmost bit is the most significant bit; modifying it does not achieve the goal of dividing it by 2; rather, it alters the scale of the number. What we want is to eliminate the least significant bit. One edge case could be having a single-bit input, where the machine must check the string length so it doesn't just make the string blank.

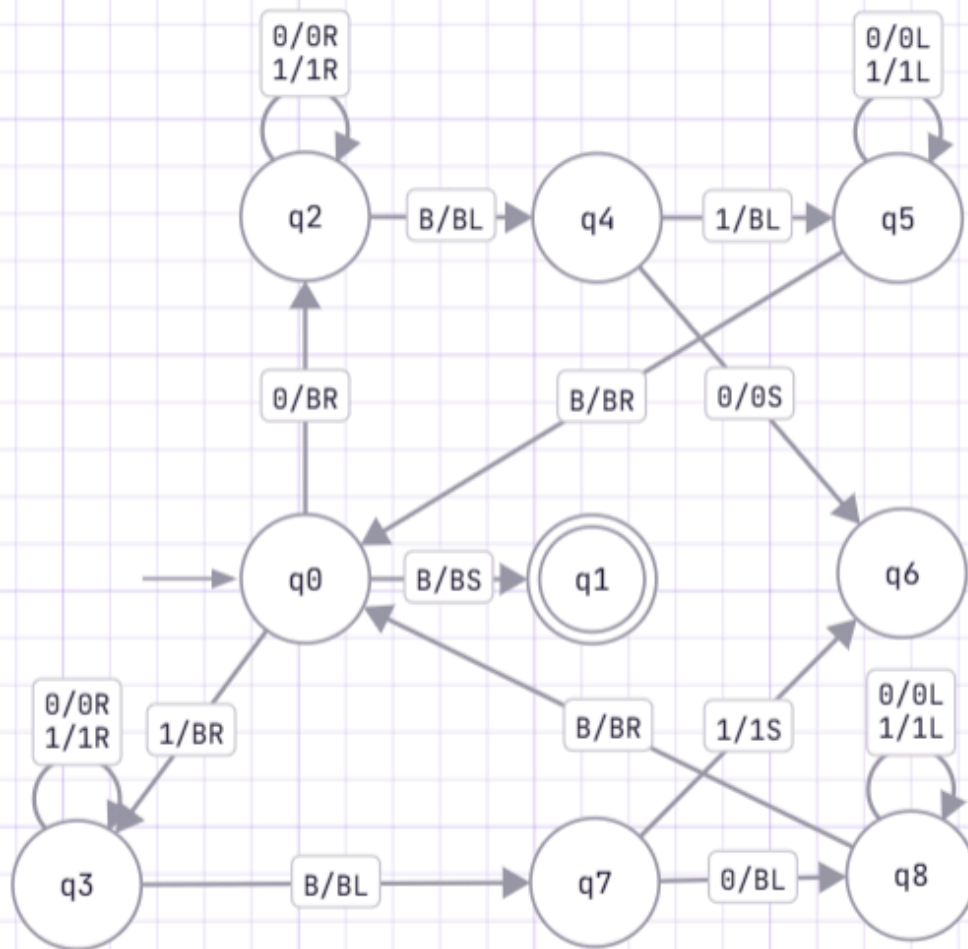
Question 4:

TaskA:

For an anti-palindrome, the bits on the extreme ends should be opposite to each other. To ensure this, we first read the leftmost bit on the tape and replace it with a blank. If the bit was zero, we check for a 1 on the rightmost

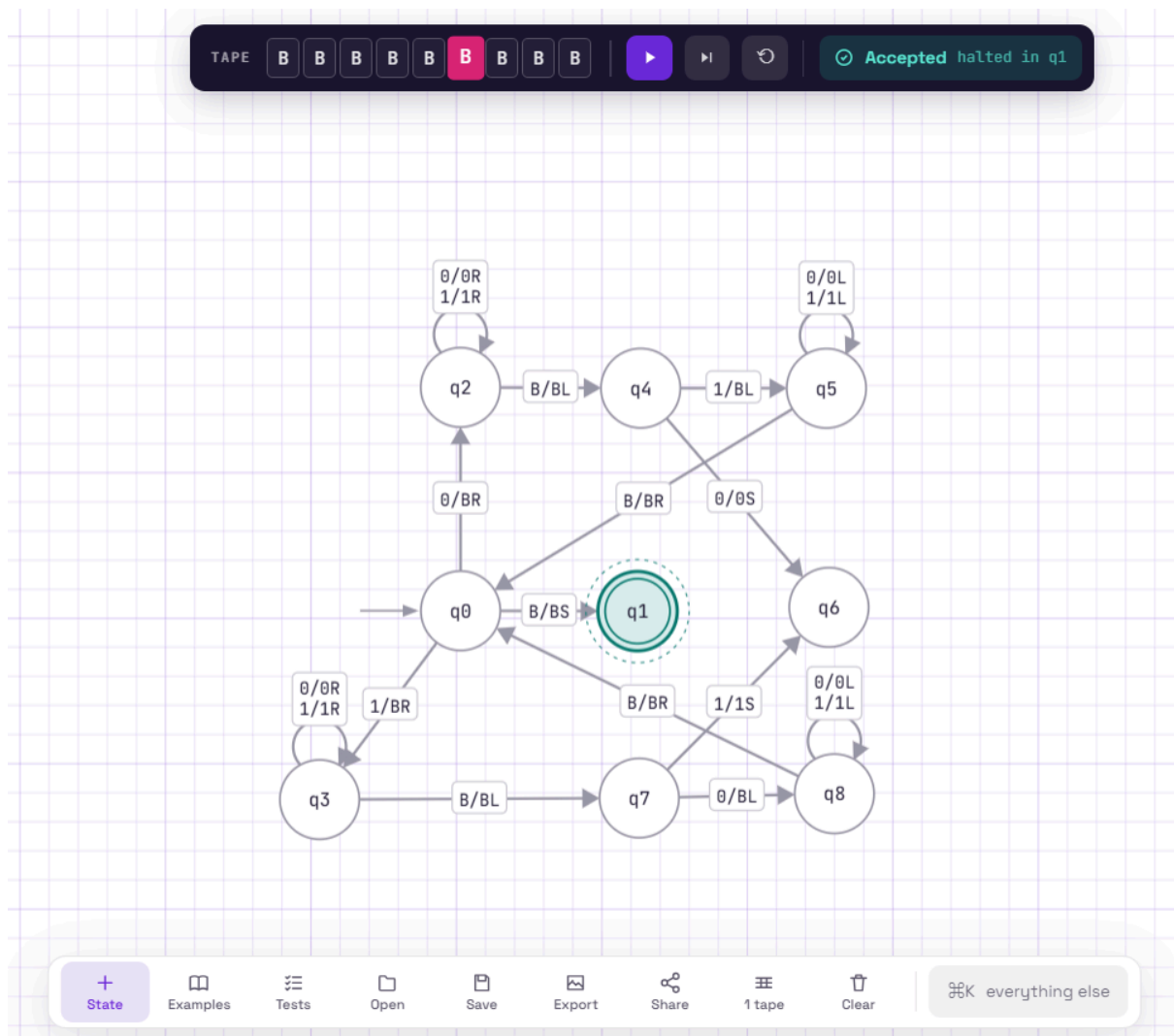
bit; if it was a one, we check for a zero on the rightmost bit. After making our leftmost bit blank, we loop to the right till we reach the blank and move to the left, reading the bit to ensure it is the opposite of the leftmost bit we just made blank. If it fulfils the condition, we replace this rightmost bit with a blank as well, move back to the leftmost bit and repeat the process until we exhaust our input. If at any point we are unable to fulfil the condition, we move to a halting state as shown in the next task, q6. This cycle continues until the input is exhausted, at which point reading an initial blank signals the machine to accept.

TaskB:



TaskC:

011001 accepted: [length exactly \$2n\$ using your personalized \$n\(n=3\)\$](#)



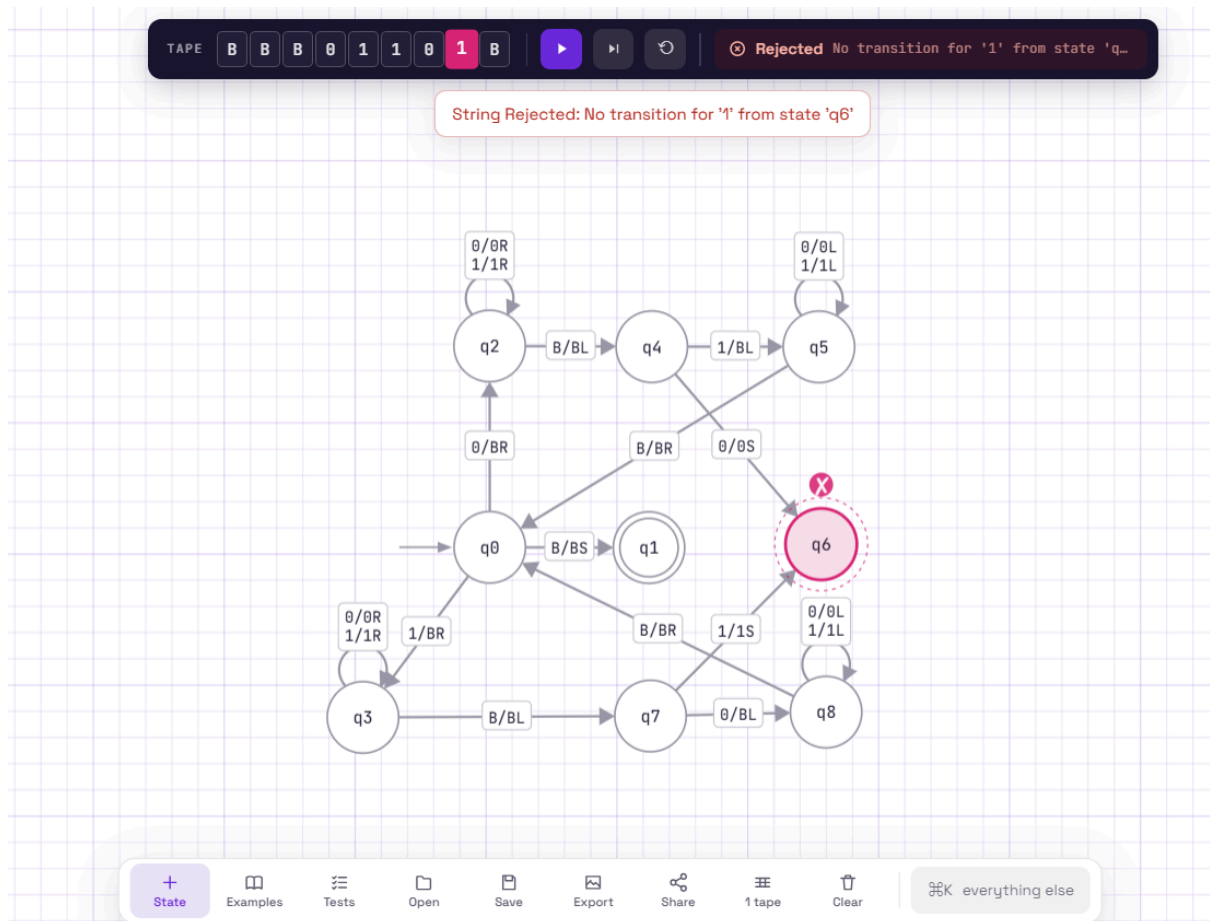
JSON: {"nodeMap":{"q0":
{"id":"q0","x":443.14416170133507,"y":214.3197233869802}
,"q1":
{"id":"q1","x":576.9534060277734,"y":214.60370954794928}
,"q2":
{"id":"q2","x":443.01800571067486,"y":45.420377725321146
},"q3":
{"id":"q3","x":336.54509109463675,"y":357.082406869041
85},"q4":
{"id":"q4","x":579.3141002998293,"y":46.01287665950749},
"q5":
{"id":"q5","x":719.765856484001,"y":46.11741180448537},"q

```

6":
{"id":"q6","x":721.6785607561736,"y":211.73991948174475},"
q7":
{"id":"q7","x":578.2268503603796,"y":355.91521545721406}
,"q8":
{"id":"q8","x":725.7984889001707,"y":352.65584113258956}
},"transitionMap":{"q0":
[{"label":"B/BS","sourceid":"q0","targetid":"q1"},
{"label":"O/BR","sourceid":"q0","targetid":"q2"},
{"label":"1/BR","sourceid":"q0","targetid":"q3"}],"q2":
[{"label":"B/BL","sourceid":"q2","targetid":"q4"},
{"label":"O/OR,1/1R","sourceid":"q2","targetid":"q2"}],"q4":
[{"label":"1/BL","sourceid":"q4","targetid":"q5"},
{"label":"O/OS","sourceid":"q4","targetid":"q6"}],"q5":
[{"label":"O/OL,1/1L","sourceid":"q5","targetid":"q5"},
{"label":"B/BR","sourceid":"q5","targetid":"q0"}],"q3":
[{"label":"B/BL","sourceid":"q3","targetid":"q7"},
{"label":"O/OR,1/1R","sourceid":"q3","targetid":"q3"}],"q7":
[{"label":"O/BL","sourceid":"q7","targetid":"q8"},
{"label":"1/1S","sourceid":"q7","targetid":"q6"}],"q8":
[{"label":"O/OL,1/1L","sourceid":"q8","targetid":"q8"},
{"label":"B/BR","sourceid":"q8","targetid":"q0"}]},"finalNodes":
["q1"],"nodeNum":9,"automataType":"TM","inputString":"01
1001","stageProps":
{"x":293.92233185783346,"y":215.30589909461276,"scale":
0.8705165431605939,"draggable":true},"stateOutput":
{},"tapeMode":"single","inputString2":""}

```

101101 rejected: [a string of length \$2n\$ that is NOT in your language](#)



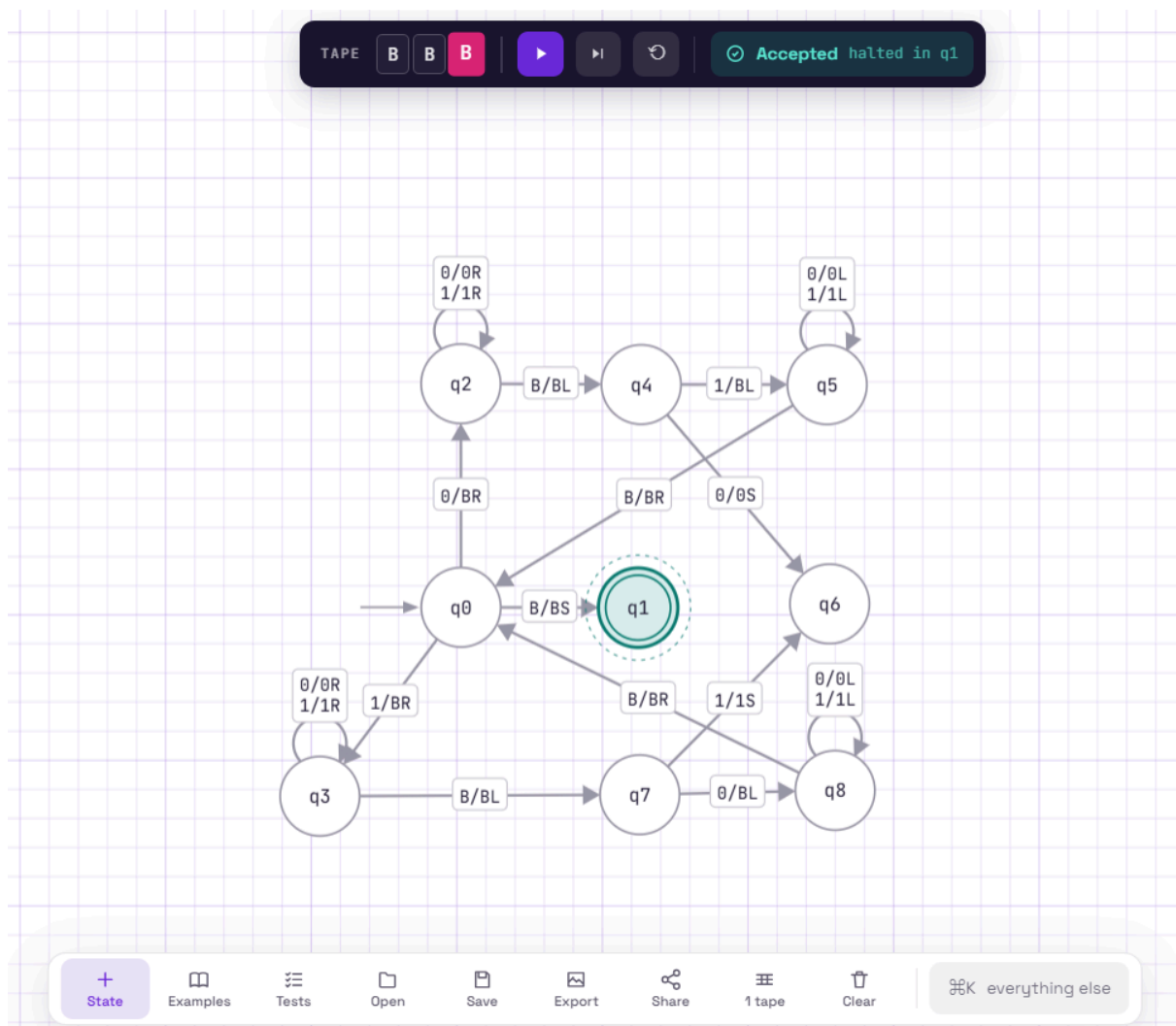
JSON: {"nodeMap":{"q0":
{"id":"q0","x":443.14416170133507,"y":214.3197233869802}
,"q1":
{"id":"q1","x":576.9534060277734,"y":214.60370954794928}
,"q2":
{"id":"q2","x":443.01800571067486,"y":45.420377725321146
},"q3":
{"id":"q3","x":336.54509109463675,"y":357.082406869041
85},"q4":
{"id":"q4","x":579.3141002998293,"y":46.01287665950749},
"q5":
{"id":"q5","x":719.765856484001,"y":46.11741180448537},"q
6":
{"id":"q6","x":721.6785607561736,"y":211.73991948174475},"
q7":

```

{"id":"q7","x":578.2268503603796,"y":355.91521545721406}
,"q8":
{"id":"q8","x":725.7984889001707,"y":352.65584113258956}
},"transitionMap":{"q0":
[{"label":"B/BS","sourceid":"q0","targetid":"q1"},
{"label":"O/BR","sourceid":"q0","targetid":"q2"},
{"label":"1/BR","sourceid":"q0","targetid":"q3"}],"q2":
[{"label":"B/BL","sourceid":"q2","targetid":"q4"},
{"label":"O/OR,1/1R","sourceid":"q2","targetid":"q2"}],"q4":
[{"label":"1/BL","sourceid":"q4","targetid":"q5"},
{"label":"O/OS","sourceid":"q4","targetid":"q6"}],"q5":
[{"label":"O/OL,1/1L","sourceid":"q5","targetid":"q5"},
{"label":"B/BR","sourceid":"q5","targetid":"q0"}],"q3":
[{"label":"B/BL","sourceid":"q3","targetid":"q7"},
{"label":"O/OR,1/1R","sourceid":"q3","targetid":"q3"}],"q7":
[{"label":"O/BL","sourceid":"q7","targetid":"q8"},
{"label":"1/1S","sourceid":"q7","targetid":"q6"}],"q8":
[{"label":"O/OL,1/1L","sourceid":"q8","targetid":"q8"},
{"label":"B/BR","sourceid":"q8","targetid":"q0"}]},"finalNodes":
["q1"],"nodeNum":9,"automataType":"TM","inputString":"10
1101","stageProps":
{"x":293.92233185783346,"y":215.30589909461276,"scale":
0.8705165431605939,"draggable":true},"stateOutput":
{},"tapeMode":"single","inputString2":""}

```

BBB accepted: [the empty string](#)



JSON: {"nodeMap":{"q0":
{"id":"q0","x":443.14416170133507,"y":214.3197233869802},
"q1":
{"id":"q1","x":576.9534060277734,"y":214.60370954794928},
"q2":
{"id":"q2","x":443.01800571067486,"y":45.420377725321146},
"q3":
{"id":"q3","x":336.54509109463675,"y":357.08240686904185},
"q4":
{"id":"q4","x":579.3141002998293,"y":46.01287665950749},
"q5":
{"id":"q5","x":719.765856484001,"y":46.11741180448537},
"q6":

```

{"id":"q6","x":721.6785607561736,"y":211.73991948174475},"
q7":
{"id":"q7","x":578.2268503603796,"y":355.91521545721406}
,"q8":
{"id":"q8","x":725.7984889001707,"y":352.65584113258956}
},"transitionMap":{"q0":
[{"label":"B/BS","sourceid":"q0","targetid":"q1"},
{"label":"O/BR","sourceid":"q0","targetid":"q2"},
{"label":"1/BR","sourceid":"q0","targetid":"q3"}],"q2":
[{"label":"B/BL","sourceid":"q2","targetid":"q4"},
{"label":"O/OR,1/1R","sourceid":"q2","targetid":"q2"}],"q4":
[{"label":"1/BL","sourceid":"q4","targetid":"q5"},
{"label":"O/OS","sourceid":"q4","targetid":"q6"}],"q5":
[{"label":"O/OL,1/1L","sourceid":"q5","targetid":"q5"},
{"label":"B/BR","sourceid":"q5","targetid":"q0"}],"q3":
[{"label":"B/BL","sourceid":"q3","targetid":"q7"},
{"label":"O/OR,1/1R","sourceid":"q3","targetid":"q3"}],"q7":
[{"label":"O/BL","sourceid":"q7","targetid":"q8"},
{"label":"1/1S","sourceid":"q7","targetid":"q6"}],"q8":
[{"label":"O/OL,1/1L","sourceid":"q8","targetid":"q8"},
{"label":"B/BR","sourceid":"q8","targetid":"q0"}]},"finalNode
s":
["q1"],"nodeNum":9,"automataType":"TM","inputString":"","s
tageProps":
{"x":293.92233185783346,"y":215.30589909461276,"scale":
0.8705165431605939,"draggable":true},"stateOutput":
{},"tapeMode":"single","inputString2":""}

```

TaskD:

A single left-to-right pass fails because the machine doesn't have the memory capacity to store the first half of

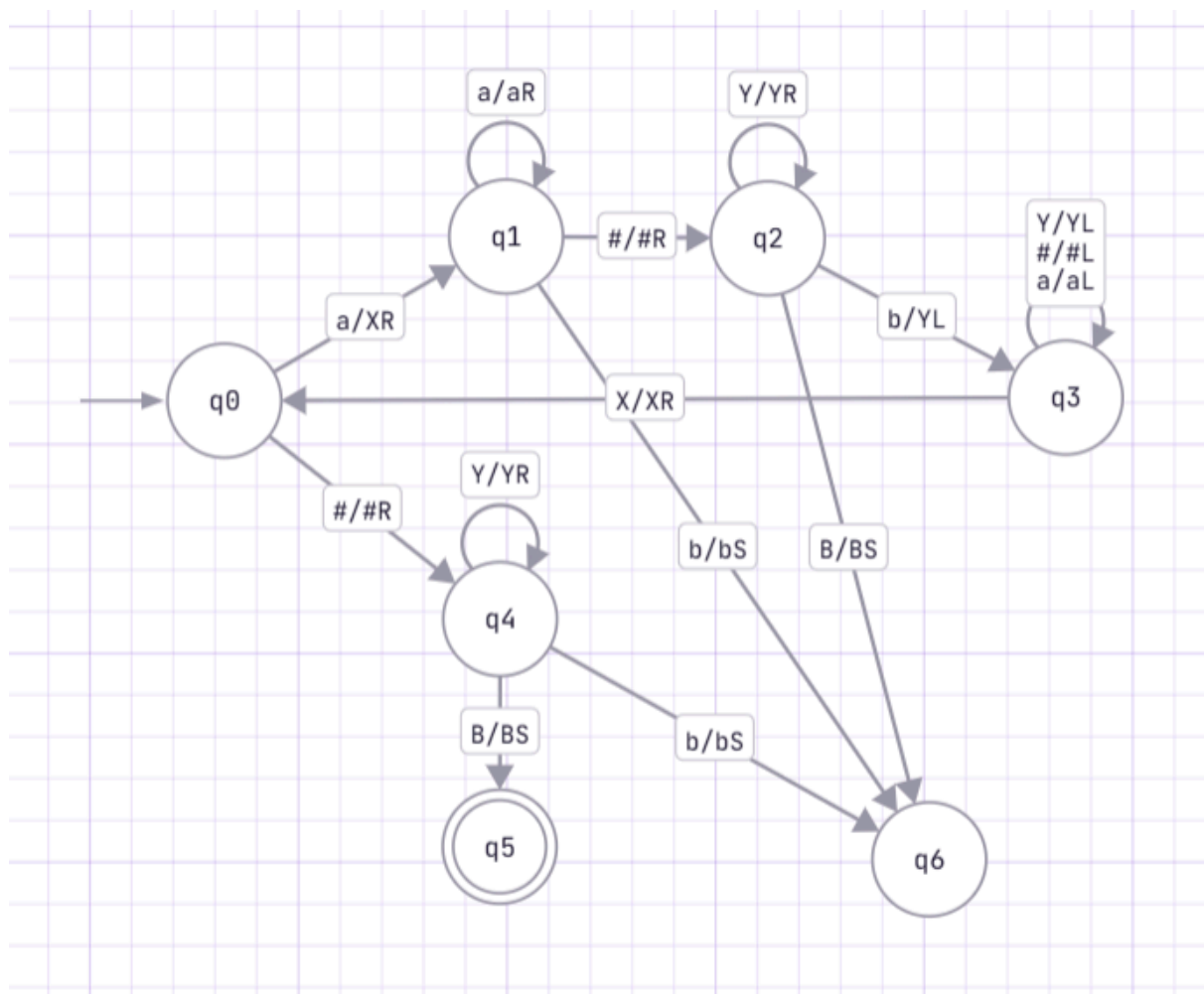
the string to compare with the second half. Thus, it must move back and forth to verify each pair one at a time.

Question 5:

TaskA:

The machine reads the tape from left to right. Our tape contains a string of $\{a^i \# b^j : i = j\}$; we need to make sure our number of a's equals the number of b's, with a special condition that they need to be separated by a #. We first read the leftmost bit a and replace it with X; then we move to the right until we reach #; then we loop to find our b and mark it with Y, after which we loop left until we reach our marked X and again check for a by moving right and doing the same process. When we have modifying all a we move to a new state and run for the last time if we have an extra b we enter reject state eg q6, else move to accept state if we reach the blank space marking the end if no extra b found.

TaskB:



TaskC:

My variant tells me $(i = j)$, and my $B = 1011011$

I need to convert my 1s to a and 0s to B to build the left-hand side of the formula, which is $(a^i \# b^j)$

So, for $B = 1011011$, our unary is abaabaa.

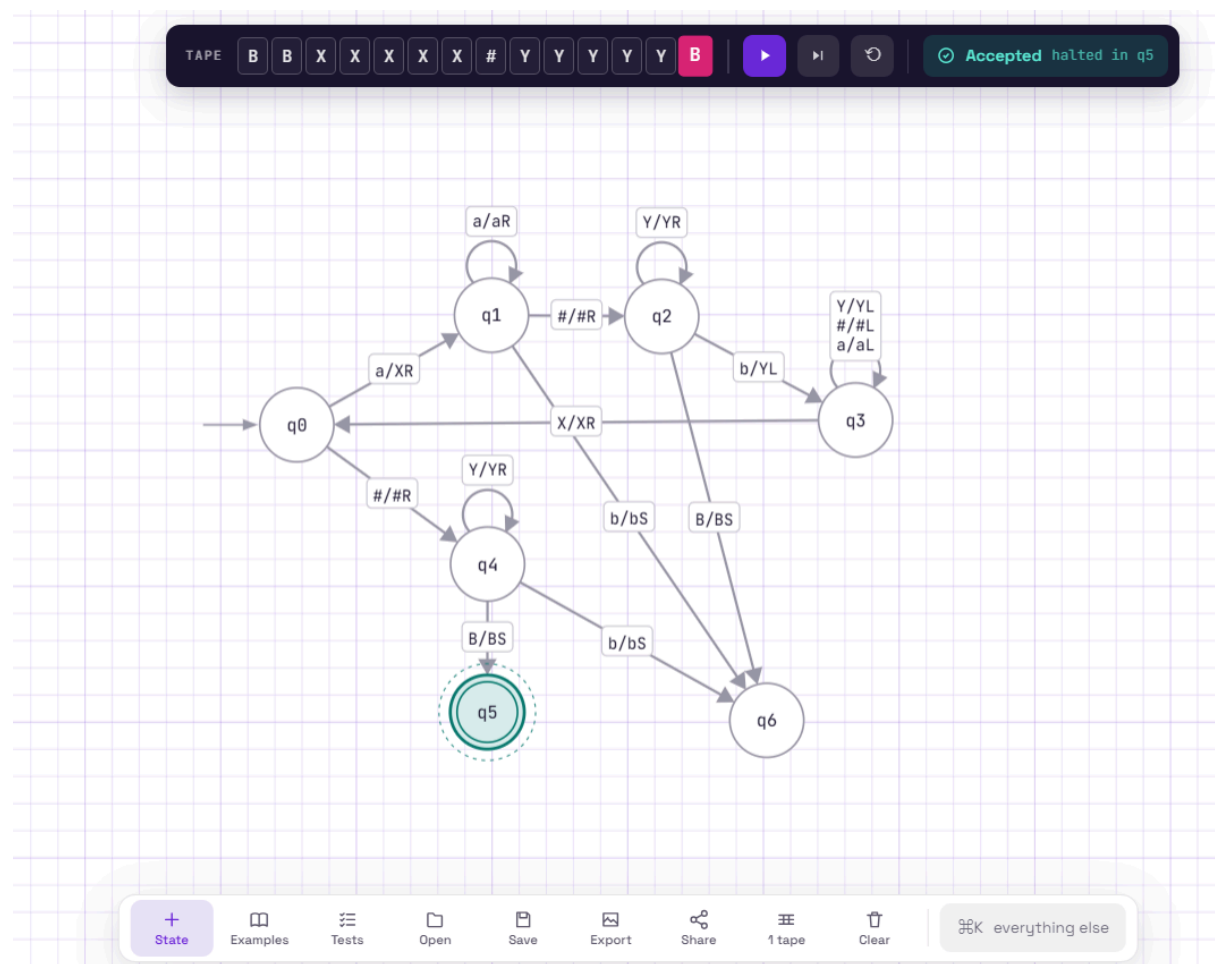
This contradicts our formula, which tells us only i number of a's are accepted on the left side of the formula, while we have a combination of a's and b's.

To fulfil my condition, I only take the a's from the unary string and remove the b's, which would make the string abaabaa into aaaaaa.

Now, to make our right side, we need two paths: one that ends up in the reject state and the other in the accept

state. So, for our accept state, we'll keep the same number of b's as our a's so our input would be: aaaaa#bbbbbb
 For our reject path, we keep using the input as: aaaaa#bb.

[Input Accepted aaaaa#bbbbbb](#)



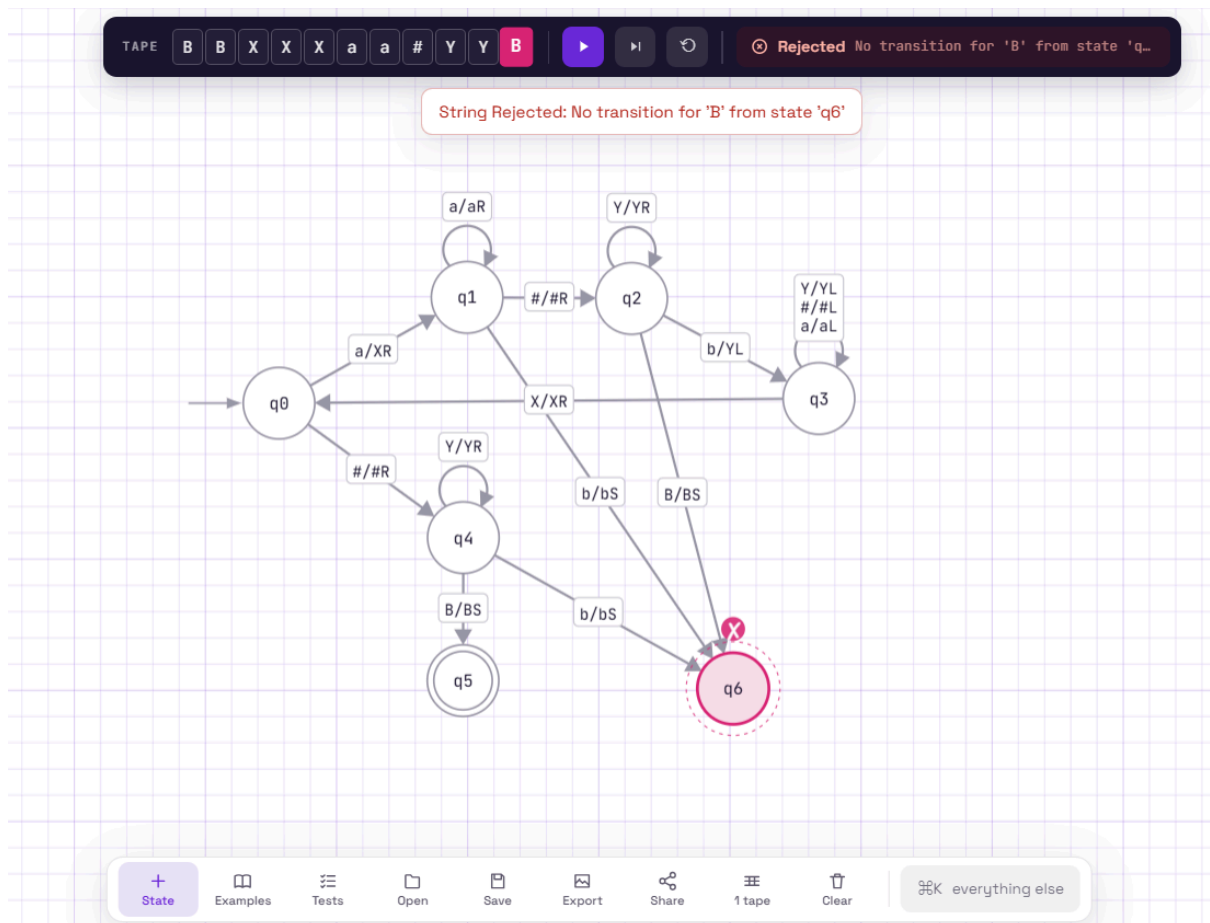
JSON: {"nodeMap":{"q0":
 {"id":"q0","x":391.7306371532072,"y":89.2596337571465},"q
 1":
 {"id":"q1","x":549.5140259997845,"y":0.5575071900599653
 }, "q2":
 {"id":"q2","x":687.5276765160755,"y":1.5136525314708251},
 "q3":
 {"id":"q3","x":844.6681732116997,"y":85.32248282613992},"
 q4":

```

{"id":"q4","x":546.3543550110469,"y":201.9777926099666
3},"q5":
{"id":"q5","x":546.1087547402454,"y":321.8305859090127},
"q6":
{"id":"q6","x":772.6959460282887,"y":328.49684715532015}
},"transitionMap":{"q0":
[{"label":"a/XR","sourceid":"q0","targetid":"q1"},
{"label":"#/#R","sourceid":"q0","targetid":"q4"}],"q1":
[{"label":"a/aR","sourceid":"q1","targetid":"q1"},
{"label":"#/#R","sourceid":"q1","targetid":"q2"},
{"label":"b/bS","sourceid":"q1","targetid":"q6"}],"q2":
[{"label":"Y/YR","sourceid":"q2","targetid":"q2"},
{"label":"b/YL","sourceid":"q2","targetid":"q3"},
{"label":"B/BS","sourceid":"q2","targetid":"q6"}],"q3":
[{"label":"Y/YL,#/#L,a/aL","sourceid":"q3","targetid":"q3"},
{"label":"X/XR","sourceid":"q3","targetid":"q0"}],"q4":
[{"label":"Y/YR","sourceid":"q4","targetid":"q4"},
{"label":"b/bS","sourceid":"q4","targetid":"q6"},
{"label":"B/BS","sourceid":"q4","targetid":"q5"}]},"finalNode
s":
["q5"],"nodeNum":7,"automataType":"TM","inputString":"aa
aaa#bbbbbb","stageProps":
{"x":192.92233185783346,"y":220.30589909461276,"scale":
0.8705165431605939,"draggable":true},"stateOutput":
{},"tapeMode":"single","inputString2":""}

```

[Input Rejected aaaaa#bb](#)



```

JSON: {"nodeMap":{"q0":
{"id":"q0","x":391.7306371532072,"y":89.2596337571465},"q
1":
{"id":"q1","x":549.5140259997845,"y":0.5575071900599653
},"q2":
{"id":"q2","x":687.5276765160755,"y":1.5136525314708251},
"q3":
{"id":"q3","x":844.6681732116997,"y":85.32248282613992},"
q4":
{"id":"q4","x":546.3543550110469,"y":201.9777926099666
3},"q5":
{"id":"q5","x":546.1087547402454,"y":321.8305859090127},
"q6":
{"id":"q6","x":772.6959460282887,"y":328.49684715532015}
},"transitionMap":{"q0":

```

```
[{"label":"a/XR","sourceid":"q0","targetid":"q1"},
{"label":"#/#R","sourceid":"q0","targetid":"q4"}], "q1":
[{"label":"a/aR","sourceid":"q1","targetid":"q1"},
{"label":"#/#R","sourceid":"q1","targetid":"q2"},
{"label":"b/bS","sourceid":"q1","targetid":"q6"}], "q2":
[{"label":"Y/YR","sourceid":"q2","targetid":"q2"},
{"label":"b/YL","sourceid":"q2","targetid":"q3"},
{"label":"B/BS","sourceid":"q2","targetid":"q6"}], "q3":
[{"label":"Y/YL,#/#L,a/aL","sourceid":"q3","targetid":"q3"},
{"label":"X/XR","sourceid":"q3","targetid":"q0"}], "q4":
[{"label":"Y/YR","sourceid":"q4","targetid":"q4"},
{"label":"b/bS","sourceid":"q4","targetid":"q6"},
{"label":"B/BS","sourceid":"q4","targetid":"q5"}]], "finalNodes":
["q5"], "nodeNum":7, "automataType":"TM", "inputString":"aa
aaa#bb", "stageProps":
{"x":192.92233185783346, "y":220.30589909461276, "scale":
0.8705165431605939, "draggable":true}, "stateOutput":
{}, "tapeMode":"single", "inputString2":""}
```

TaskD:

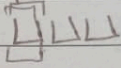
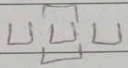
The Turing machine does not have memory to be able to remember all the inputs it had read and the number of a's it had counted to be able to compare it with the number of b's. One method to solve this would be using multiple states for each next a, but it would make our machine have many states, which isn't possible. So the best ideal solution is to move back and forth, comparing one a and one b at a time until all a's or all b's are exhausted, to give our final output of where it lands, either in an accept or reject state.

Question 6:

TaskA:

For empty string input:

q = accept

Step	State	Tape & Head Position	Transition
Start	q _{-even}		Read 1 → Write 1 Move R go to q _{-odd}
1	q _{-odd}		Read 1 → Write 1 Move R go to q _{-reject}
2	q _{-reject}		Machine stops Halt

For input string "11":

Date _____ 20__

For input "11"

Step

Step	State	Tap & Head Position	Transition
start	q _{even}		Read 1 → write 1 Move R go to q _{odd}
1	q _{odd}		Read 1 → write 1 Move R go to q _{even}
2	q _{even}		Read □ → write □ Move R go to q _{odd}
3	q _{odd}		Read □ → write □ Move R go to q _{reject}

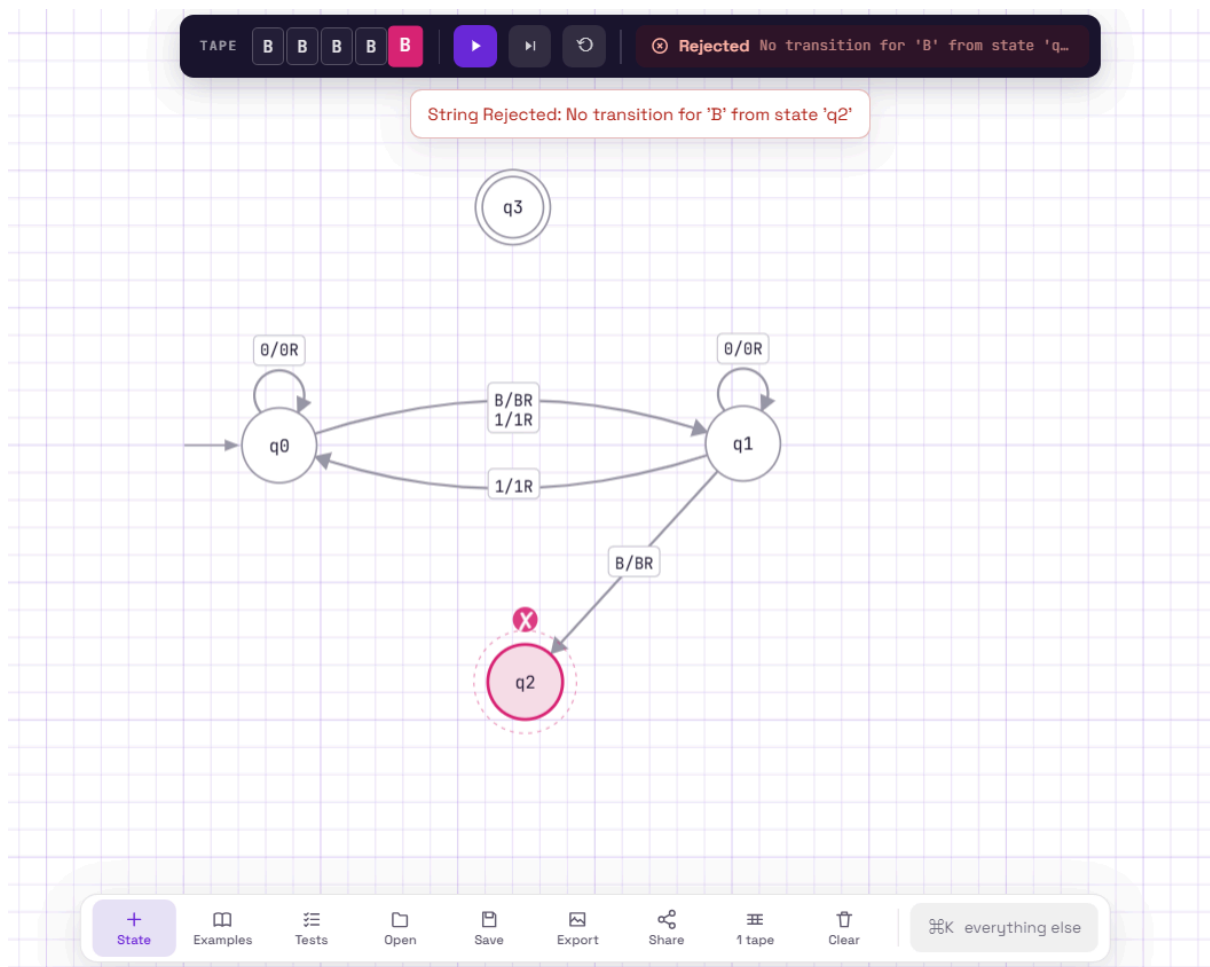
TaskB:

The fault lies in row 3. When the state is q_{even} and reads a blank (denoted by □), it should write □, move R, and transition to the q_{accept} state. Currently, the table shows that q_{even} moves to q_{odd} upon reading a blank. As shown in my hand traces, this exact row is what caused valid inputs—both the empty string and the string "11"—to be incorrectly routed into q_{reject} at the very last step. Fixing this row to point to q_{accept} resolves the issue.

TaskC:

Broken Machine:

=> Input String ""



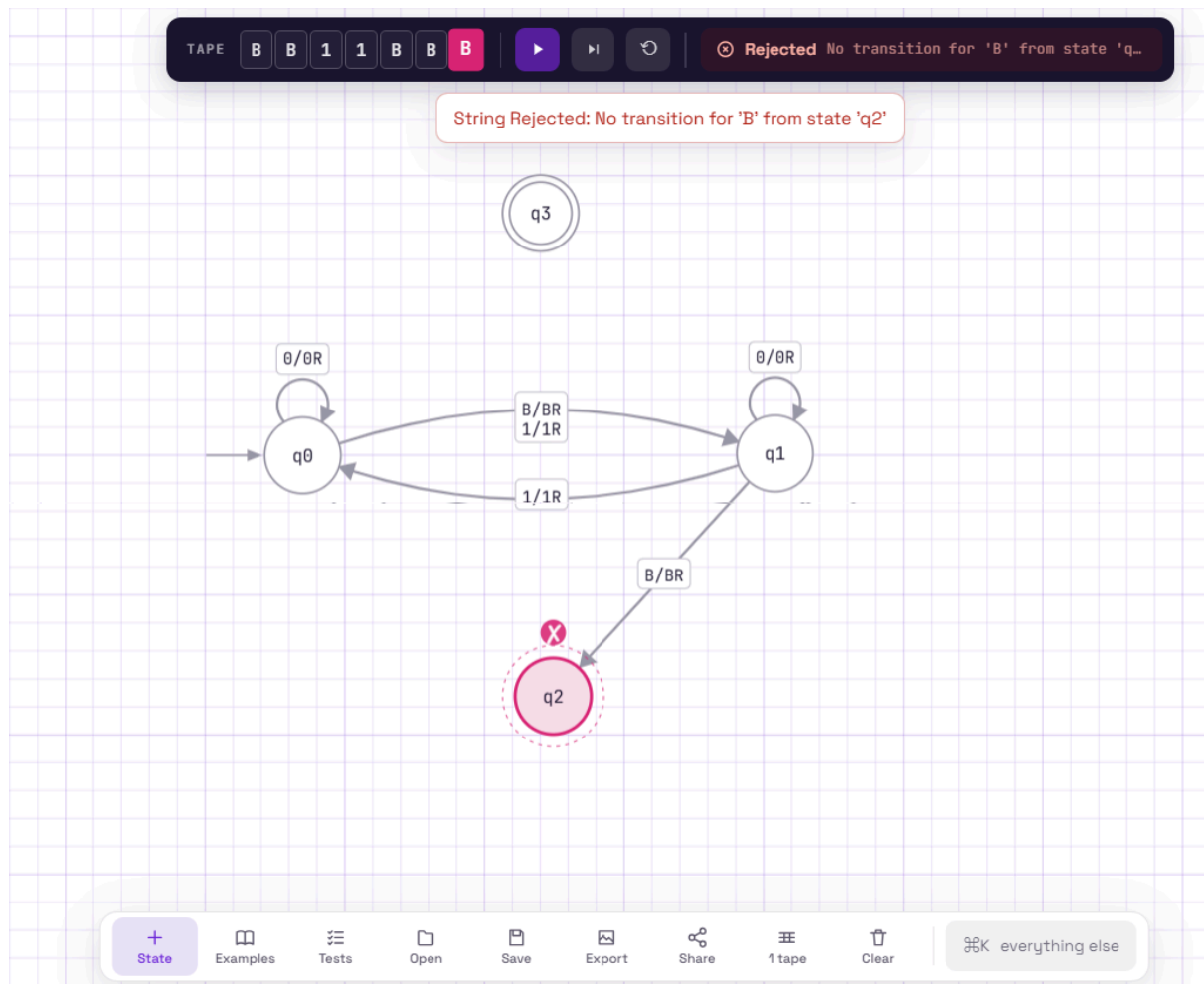
```
JSON: {"nodeMap":{"q0":
{"id":"q0","x":406.8448174171307,"y":110.18187004394775},
"q1":
{"id":"q1","x":778.9020138778659,"y":108.74674761557753},
"q2":
{"id":"q2","x":604.1476819396144,"y":299.6605588034809
7},"q3":
{"id":"q3","x":594.2890196230293,"y":-80.47953917342099
}}, "transitionMap":{"q0":
[{"label":"0/0R","sourceid":"q0","targetid":"q0"},
{"label":"B/BR,1/1R","sourceid":"q0","targetid":"q1"}],"q1":
[{"label":"0/0R","sourceid":"q1","targetid":"q1"},
{"label":"1/1R","sourceid":"q1","targetid":"q0"},
{"label":"B/BR","sourceid":"q1","targetid":"q2"}],"finalNode
```

s":

```
["q3"],"nodeNum":4,"automataType":"TM","inputString":"","stageProps":
```

```
{"x":192.92233185783346,"y":220.30589909461276,"scale":0.8705165431605939,"draggable":true},"stateOutput":{},"tapeMode":"single","inputString2":""}
```

=> [Input String "11"](#)



JSON: {"nodeMap":{"q0":

{"id":"q0","x":406.8448174171307,"y":110.18187004394775},

"q1":

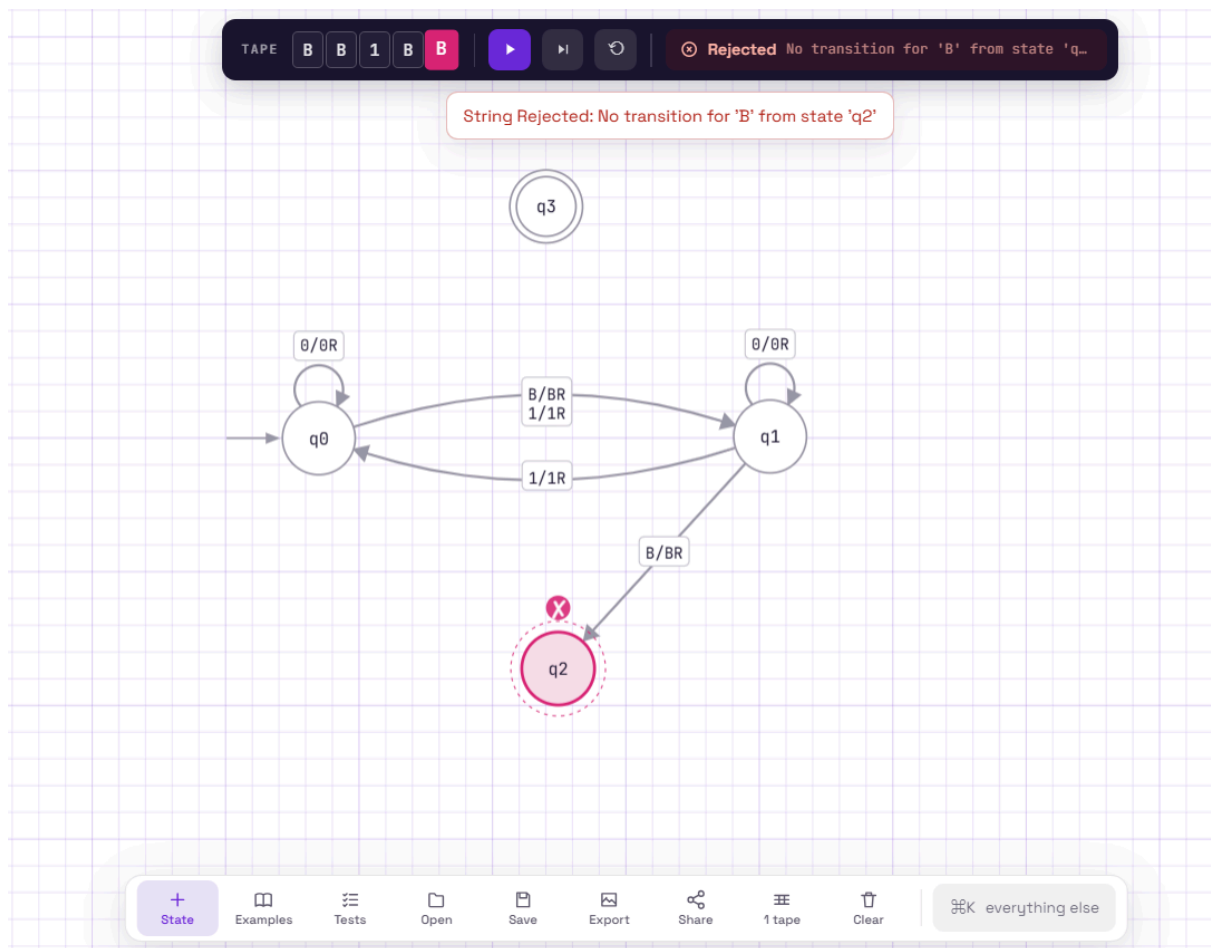
{"id":"q1","x":778.9020138778659,"y":108.74674761557753},"

q2":

{"id":"q2","x":604.1476819396144,"y":299.6605588034809

```
7},"q3":  
{ "id": "q3", "x": 594.2890196230293, "y": -80.47953917342099  
}, "transitionMap": { "q0":  
[ { "label": "O/OR", "sourceid": "q0", "targetid": "q0" },  
{ "label": "B/BR,1/1R", "sourceid": "q0", "targetid": "q1" } ], "q1":  
[ { "label": "O/OR", "sourceid": "q1", "targetid": "q1" },  
{ "label": "1/1R", "sourceid": "q1", "targetid": "q0" },  
{ "label": "B/BR", "sourceid": "q1", "targetid": "q2" } ] }, "finalNodes":  
[ "q3" ], "nodeNum": 4, "automataType": "TM", "inputString": "11",  
"stageProps":  
{ "x": 192.92233185783346, "y": 220.30589909461276, "scale":  
0.8705165431605939, "draggable": true }, "stateOutput":  
{ }, "tapeMode": "single", "inputString2": "" }
```

=> [Input String “1”](#)

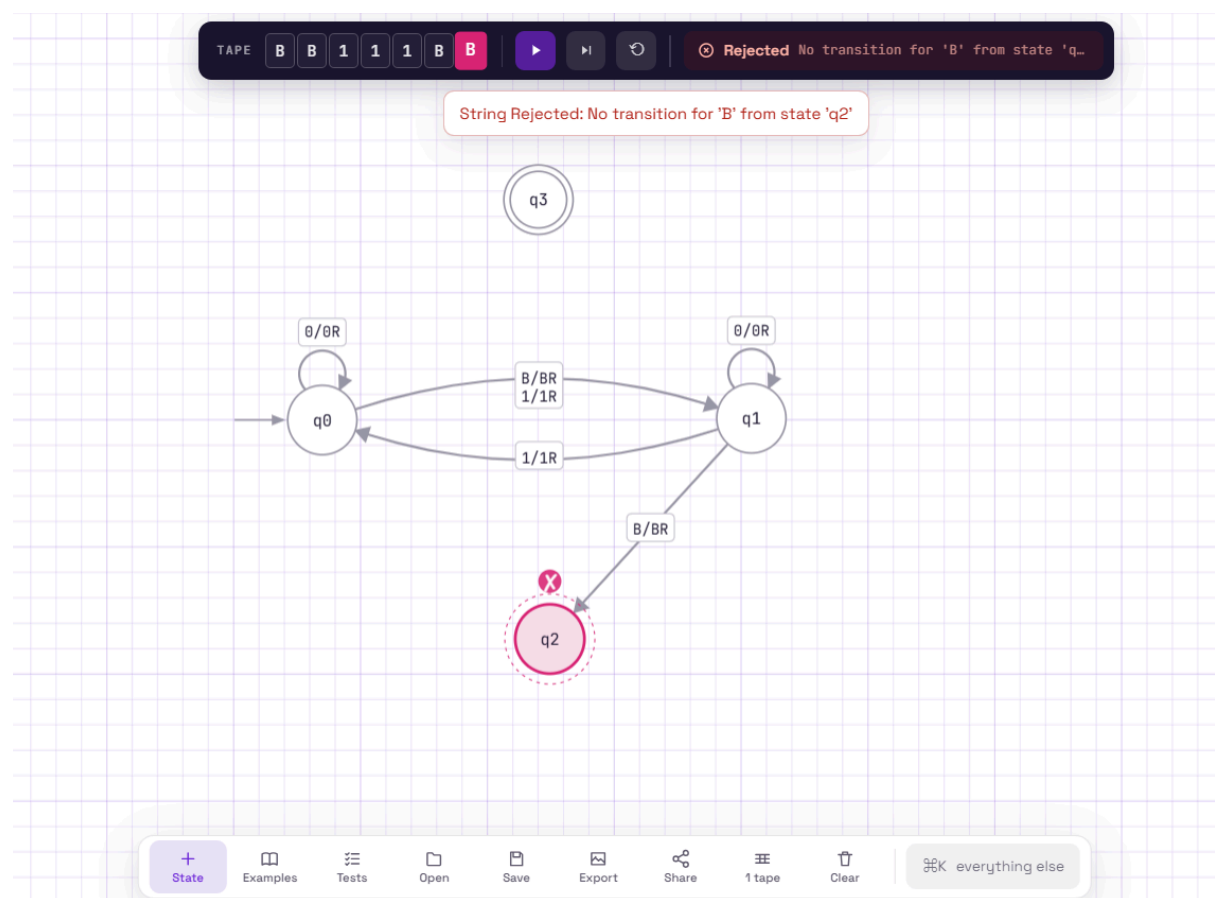


```
JSON: {"nodeMap":{"q0":
{"id":"q0","x":406.8448174171307,"y":110.18187004394775},
"q1":
{"id":"q1","x":778.9020138778659,"y":108.74674761557753},
"q2":
{"id":"q2","x":604.1476819396144,"y":299.6605588034809
7},"q3":
{"id":"q3","x":594.2890196230293,"y":-80.47953917342099
}}, "transitionMap":{"q0":
[{"label":"0/OR","sourceid":"q0","targetid":"q0"},
{"label":"B/BR,1/1R","sourceid":"q0","targetid":"q1"}],"q1":
[{"label":"0/OR","sourceid":"q1","targetid":"q1"},
{"label":"1/1R","sourceid":"q1","targetid":"q0"},
{"label":"B/BR","sourceid":"q1","targetid":"q2"}]},"finalNode
```

s":

```
[{"q3"},"nodeNum":4,"automataType":"TM","inputString":"1",  
"stageProps":  
{ "x":192.92233185783346,"y":220.30589909461276,"scale":  
0.8705165431605939,"draggable":true},"stateOutput":  
{},"tapeMode":"single","inputString2":""}
```

=> [Input String "111"](#)

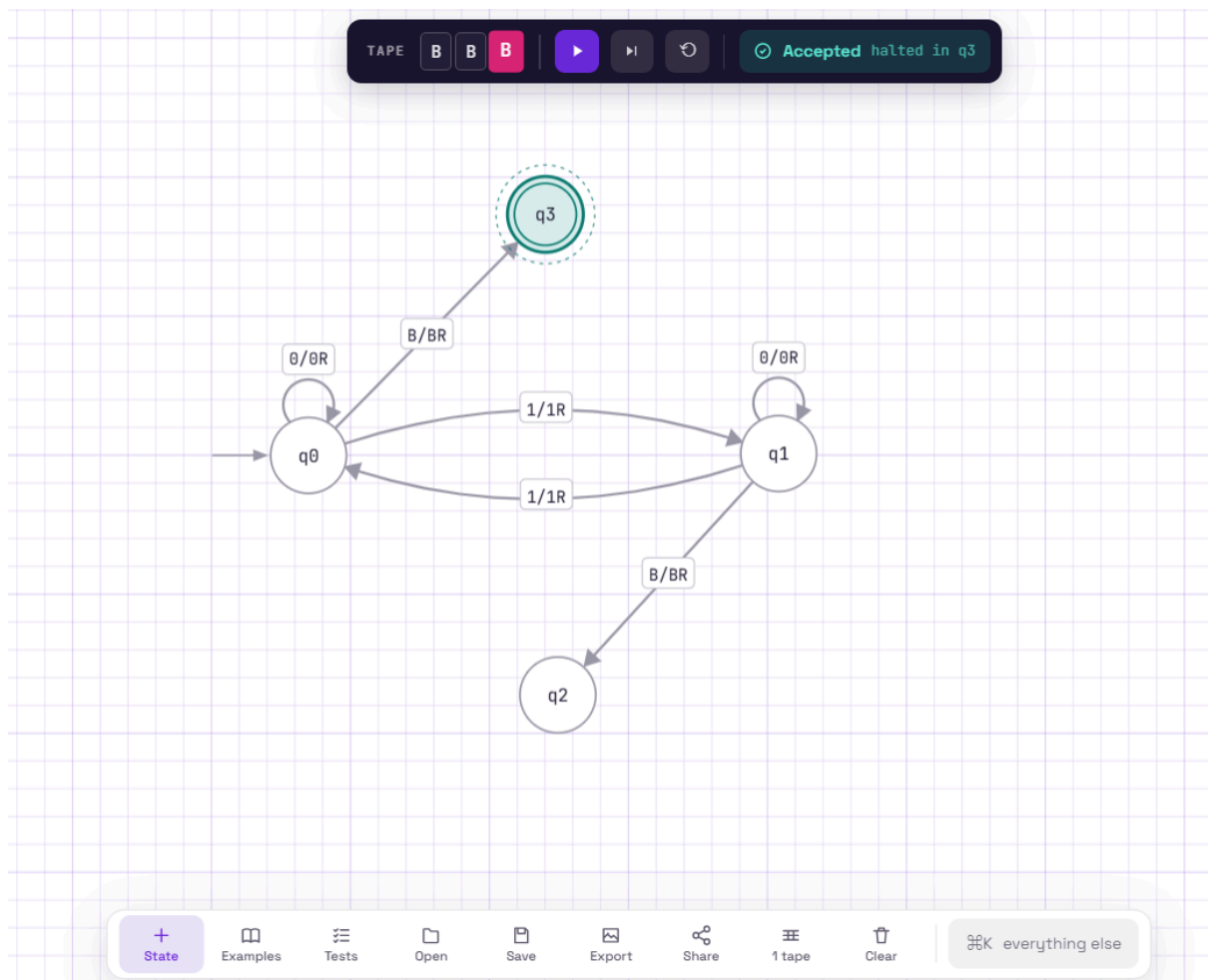


JSON: {"nodeMap":{"q0":
{"id":"q0","x":406.8448174171307,"y":110.18187004394775},
"q1":
{"id":"q1","x":778.9020138778659,"y":108.74674761557753},
"q2":
{"id":"q2","x":604.1476819396144,"y":299.6605588034809
7},"q3":

```
{"id":"q3","x":594.2890196230293,"y":-80.47953917342099
}}, "transitionMap": {"q0":
[{"label":"O/OR","sourceid":"q0","targetid":"q0"},
{"label":"B/BR,1/1R","sourceid":"q0","targetid":"q1"}], "q1":
[{"label":"O/OR","sourceid":"q1","targetid":"q1"},
{"label":"1/1R","sourceid":"q1","targetid":"q0"},
{"label":"B/BR","sourceid":"q1","targetid":"q2"}]}, "finalNodes":
["q3"], "nodeNum":4, "automataType":"TM", "inputString":"11
1", "stageProps":
{"x":192.92233185783346,"y":220.30589909461276,"scale":
0.8705165431605939,"draggable":true}, "stateOutput":
{}, "tapeMode":"single", "inputString2":""}
```

Corrected Machine:

=> [Input String ""](#)



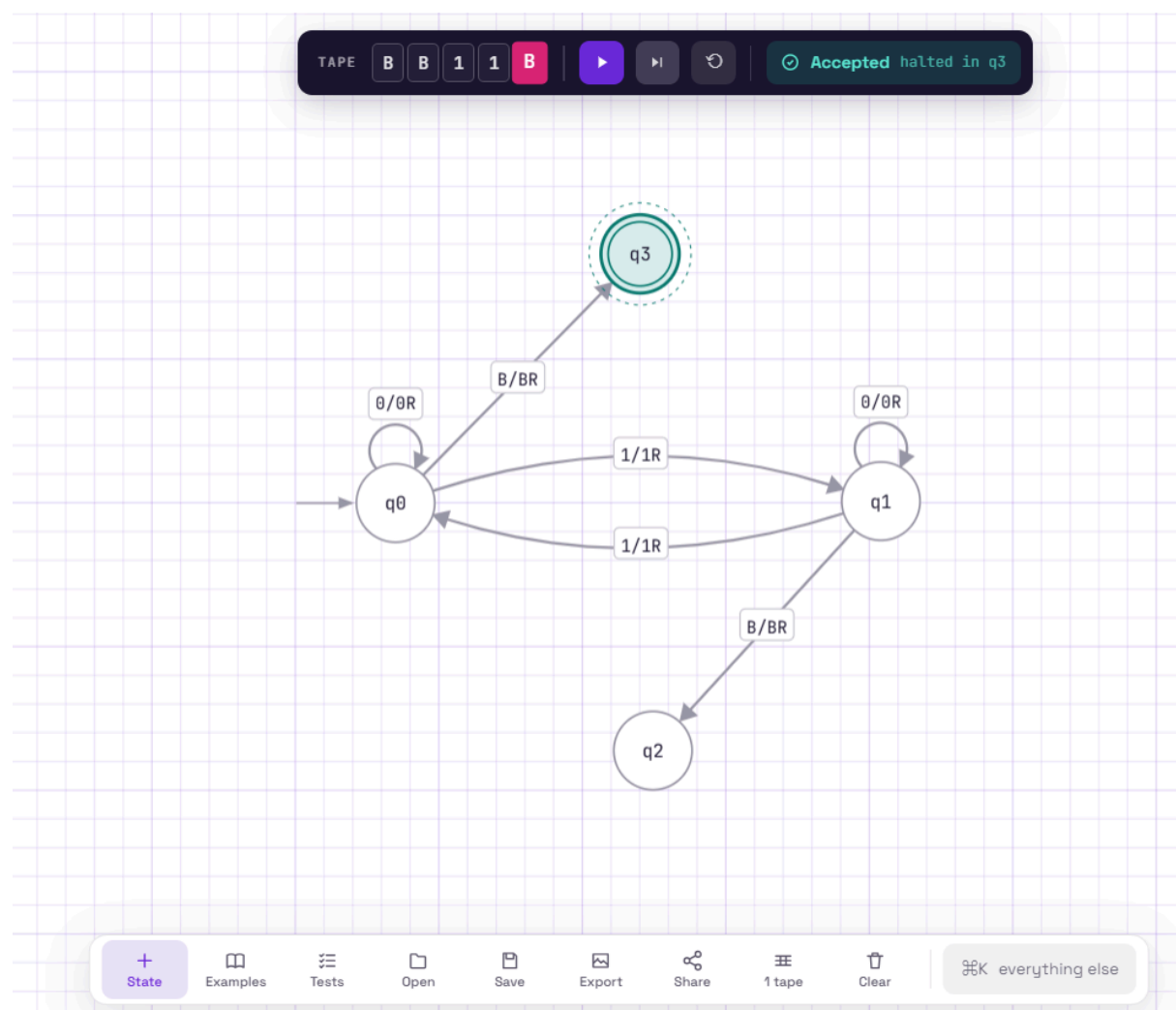
```
JSON: {"nodeMap":{"q0":
{"id":"q0","x":406.8448174171307,"y":110.18187004394775},
"q1":
{"id":"q1","x":778.9020138778659,"y":108.74674761557753},
"q2":
{"id":"q2","x":604.1476819396144,"y":299.6605588034809
7},"q3":
{"id":"q3","x":594.2890196230293,"y":-80.47953917342099
}}, "transitionMap":{"q0":
[{"label":"0/0R","sourceid":"q0","targetid":"q0"},
{"label":"1/1R","sourceid":"q0","targetid":"q1"},
{"label":"B/BR","sourceid":"q0","targetid":"q3"}],"q1":
[{"label":"0/0R","sourceid":"q1","targetid":"q1"},
{"label":"1/1R","sourceid":"q1","targetid":"q0"},
{"label":"B/BR","sourceid":"q1","targetid":"q2"}], "q2":
[{"label":"1/1R","sourceid":"q2","targetid":"q1"}], "q3":
[]}}
```

```

{"label":"B/BR","sourceid":"q1","targetid":"q2"}]], "finalNodes":
["q3"], "nodeNum": 4, "automataType": "TM", "inputString": "",
stageProps":
{"x": 192.92233185783346, "y": 220.30589909461276, "scale":
0.8705165431605939, "draggable": true}, "stateOutput":
{}, "tapeMode": "single", "inputString2": ""}

```

=> [Input String "11"](#)



```

JSON: {"nodeMap":{"q0":
{"id":"q0","x":406.8448174171307,"y":110.18187004394775},
"q1":
{"id":"q1","x":778.9020138778659,"y":108.74674761557753},

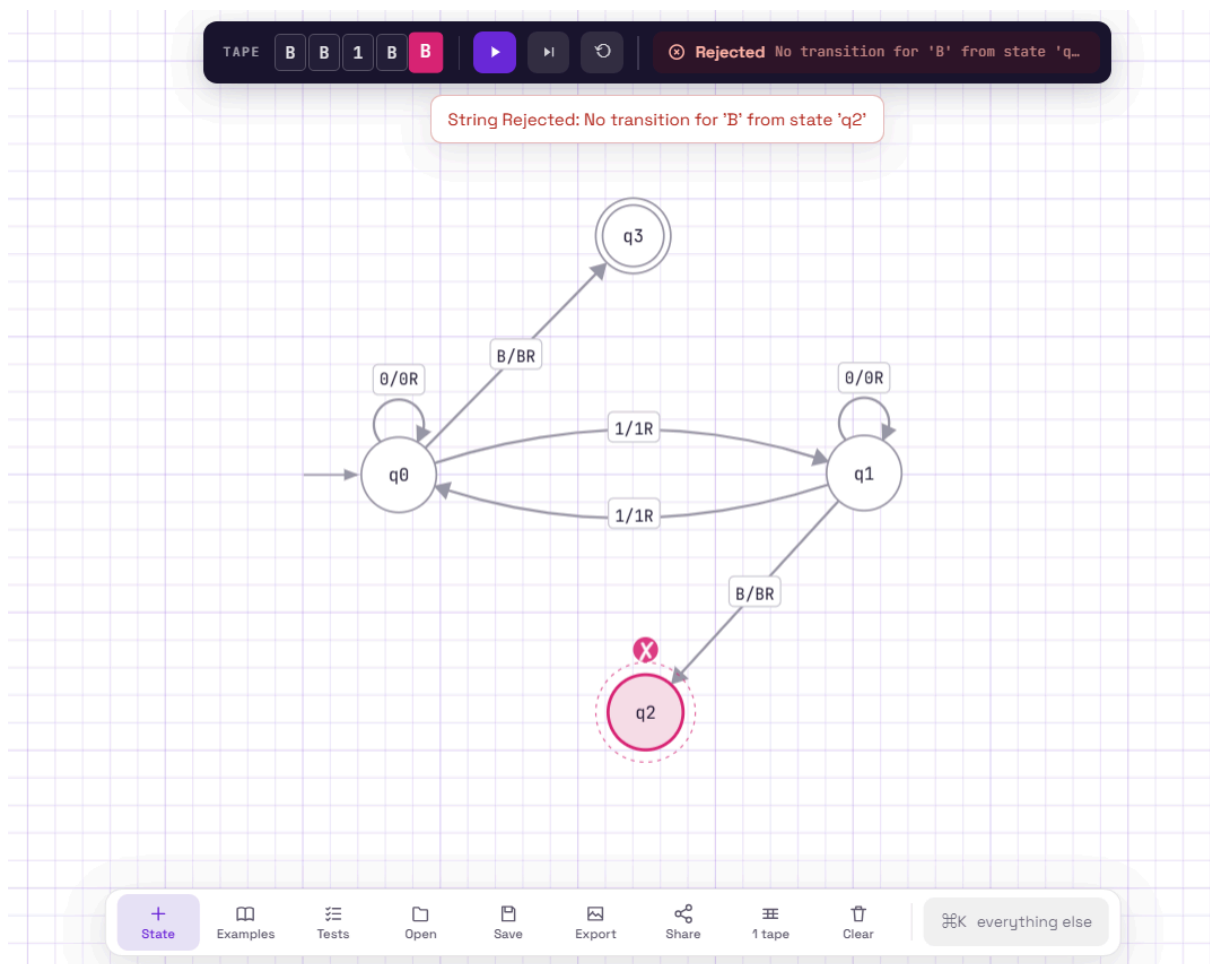
```

```

q2":
{"id":"q2","x":604.1476819396144,"y":299.6605588034809
7},"q3":
{"id":"q3","x":594.2890196230293,"y":-80.47953917342099
}}, "transitionMap": {"q0":
[{"label":"O/OR","sourceid":"q0","targetid":"q0"},
{"label":"1/1R","sourceid":"q0","targetid":"q1"},
{"label":"B/BR","sourceid":"q0","targetid":"q3"}], "q1":
[{"label":"O/OR","sourceid":"q1","targetid":"q1"},
{"label":"1/1R","sourceid":"q1","targetid":"q0"},
{"label":"B/BR","sourceid":"q1","targetid":"q2"}]}, "finalNode
s":
["q3"], "nodeNum":4, "automataType":"TM", "inputString":"11
", "stageProps":
{"x":255.71124203096872,"y":238.90744239705552,"scale":
0.8857805000000003,"draggable":true}, "stateOutput":
{}, "tapeMode":"single", "inputString2":""}

```

=> [Input String “1”](#)



```

JSON: {"nodeMap":{"q0":
{"id":"q0","x":406.8448174171307,"y":110.18187004394775},
"q1":
{"id":"q1","x":778.9020138778659,"y":108.74674761557753},
"q2":
{"id":"q2","x":604.1476819396144,"y":299.6605588034809
7},"q3":
{"id":"q3","x":594.2890196230293,"y":-80.47953917342099
}}, "transitionMap":{"q0":
[{"label":"0/0R","sourceid":"q0","targetid":"q0"},
{"label":"1/1R","sourceid":"q0","targetid":"q1"},
{"label":"B/BR","sourceid":"q0","targetid":"q3"}],"q1":
[{"label":"0/0R","sourceid":"q1","targetid":"q1"},
{"label":"1/1R","sourceid":"q1","targetid":"q0"},

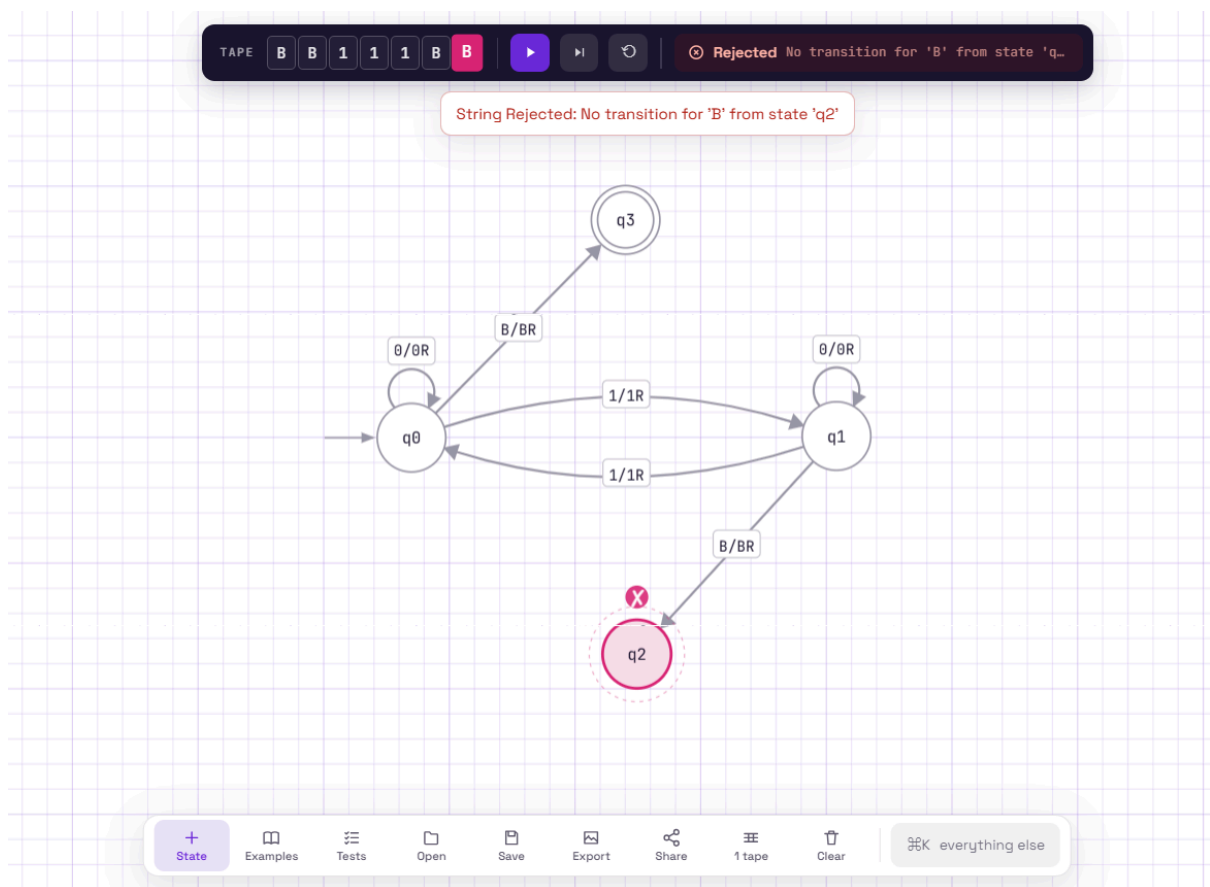
```

```

{"label":"B/BR","sourceid":"q1","targetid":"q2"}]], "finalNodes":
["q3"], "nodeNum":4, "automataType":"TM", "inputString":"1",
"stageProps":
{"x":255.71124203096872, "y":238.90744239705552, "scale":
0.88578050000000003, "draggable":true}, "stateOutput":
{}, "tapeMode":"single", "inputString2":""}

```

=> [Input String '111'](#)



```

JSON: {"nodeMap":{"q0":
{"id":"q0","x":406.8448174171307,"y":110.18187004394775},
"q1":
{"id":"q1","x":778.9020138778659,"y":108.74674761557753},
"q2":
{"id":"q2","x":604.1476819396144,"y":299.6605588034809

```

```

7},"q3":
{"id":"q3","x":594.2890196230293,"y":-80.47953917342099
}}, "transitionMap":{"q0":
[{"label":"O/OR","sourceid":"q0","targetid":"q0"},
{"label":"1/1R","sourceid":"q0","targetid":"q1"},
{"label":"B/BR","sourceid":"q0","targetid":"q3"}], "q1":
[{"label":"O/OR","sourceid":"q1","targetid":"q1"},
{"label":"1/1R","sourceid":"q1","targetid":"q0"},
{"label":"B/BR","sourceid":"q1","targetid":"q2"}]}, "finalNode
s":
["q3"], "nodeNum":4, "automataType":"TM", "inputString":"11
1", "stageProps":
{"x":255.71124203096872,"y":238.90744239705552,"scale":
0.88578050000000003,"draggable":true}, "stateOutput":
{}, "tapeMode":"single", "inputString2":""}

```

Input String	Broken Machine	Fixed Machine	Correct Answer Should Be
“”	rejected	accepted	Accepted as even number of 1s
“11”	rejected	accepted	Accepted as even number of 1s
“1”	rejected	rejected	Rejected as odd number of 1s
“111”	rejected	rejected	Rejected as odd number of 1s

TaskD:

Using any odd-length string of 1s, such as '1' or '111', would have made the machine move to the rejected state, which, in theory, we wanted, but it would hide the bug that

the machine had initially, where it rejected even-length strings of 1s too. This tells us that relying solely on one type of test case, such as where the machine should reject or accept the input, is dangerous, as we could miss the other case. So to confirm the machine is working as it is intended use balanced mix of both inputs to show that both paths works, and to also use inputs where extreme or boundary cases are checked eg; empty string for this machine.